

Q1. Searching with Heuristics

Consider the A* searching process on the connected undirected graph, with starting node S and the goal node G. Suppose the cost for each connection edge is **always positive**. We define $h^*(X)$ as the shortest (optimal) distance to G from a node X.

Answer Questions (a). **Question (b) is optional.**

(a) Suppose h is an **admissible** heuristic, and we conduct A* **tree search** using heuristic h' and finally find a solution. Let C be the cost of the found path (directed by h' , defined in part (a)) from S to G

(i) Choose **one best** answer for each condition below.

1. If $h'(X) = \frac{1}{2}h(X)$ for all Node X , then ☒ $C = h^*(S)$ ☐ $C > h^*(S)$ ☐ $C \geq h^*(S)$
2. If $h'(X) = \frac{h(X) + h^*(X)}{2}$ for all Node X , then ☒ $C = h^*(S)$ ☐ $C > h^*(S)$ ☐ $C \geq h^*(S)$
3. If $h'(X) = h(X) + h^*(X)$ for all Node X , then ☐ $C = h^*(S)$ ☐ $C > h^*(S)$ ☒ $C \geq h^*(S)$
4. If we define the set $K(X)$ for a node X as all its neighbor nodes Y satisfying $h^*(X) > h^*(Y)$, and the following always holds

$$h'(X) \leq \begin{cases} \min_{Y \in K(X)} h'(Y) - h(Y) + h(X) & \text{if } K(X) \neq \emptyset \\ h(X) & \text{if } K(X) = \emptyset \end{cases}$$

then,

☒ $C = h^*(S)$ ☐ $C > h^*(S)$ ☐ $C \geq h^*(S)$

5. If K is the same as above, we have

$$h'(X) = \begin{cases} \min_{Y \in K(X)} h(Y) + \text{cost}(X, Y) & \text{if } K(X) \neq \emptyset \\ h(X) & \text{if } K(X) = \emptyset \end{cases}$$

where $\text{cost}(X, Y)$ is the cost of the edge connecting X and Y ,
then,

☒ $C = h^*(S)$ ☐ $C > h^*(S)$ ☐ $C \geq h^*(S)$

6. If $h'(X) = \min_{Y \in K(X) + \{X\}} h(Y)$ (K is the same as above), ☒ $C = h^*(S)$ ☐ $C > h^*(S)$ ☐ $C \geq h^*(S)$

(ii) In which of the conditions above, h' is still **admissible** and for sure to dominate h ? Check all that apply. Remember we say h_1 dominates h_2 when $h_1(X) \geq h_2(X)$ holds for all X . ☐ 1 ☒ 2 ☐ 3 ☐ 4 ☐ 5 ☐ 6

(b) [Optional] Suppose h is an **admissible** heuristic, and we conduct A* **tree search** using heuristic h' and finally find a solution.

If $\epsilon > 0$, and X_0 is a node in the graph, and h' is a heuristic such that

$$h'(X) = \begin{cases} h(X) & \text{if } X = X_0 \\ h(X) + \epsilon & \text{otherwise} \end{cases}$$

- Alice claims h' can be inadmissible, and hence $C = h^*(S)$ does not always hold.
- Bob instead thinks the node expansion order directed by h' is the same as the heuristic h'' , where

$$h''(X) = \begin{cases} h(X) - \epsilon & \text{if } X = X_0 \\ h(X) & \text{if otherwise} \end{cases}$$

Since h'' is admissible and will lead to $C = h^*(S)$, and so does h' . Hence, $C = h^*(S)$ always holds.

The two conclusions (underlined) apparently contradict with each other, and **only exactly one of them are correct and the other is wrong**. Choose the **best** explanation from below - which student's conclusion is wrong, and why are they wrong?

- ☐ Alice's conclusion is wrong, because the heuristic h' is always admissible.
- ☐ Alice's conclusion is wrong, because an inadmissible heuristics does not necessarily always lead to the failure of the optimality when conducting A* tree search.
- ☐ Alice's conclusion is wrong, because of another reason that is not listed above.
- ☐ Bob's conclusion is wrong, because the node visiting expansion ordering of h'' during searching might not be the same as h' .
- ☐ Bob's conclusion is wrong, because the heuristic h'' might lead to an incomplete search, regardless of its optimally property.
- ☒ Bob's conclusion is wrong, because of another reason that is not listed above.

Choice 4 is incorrect, because the difference between h' and h'' is a constant. During searching, the choice of the expansion of the fringe will not be affected if all the nodes add the same constant to the heuristics.

Choice 5 is incorrect because there will never be an infinite loop if there are no cycle has negative COST sum (rather than HEURISTICS). If there is a cycle, such that its COST sum is positive, and all the nodes in the cycle have negative heuristics, when we do $g+h$, g is getting larger and larger, while h remains a not-that-large negative value. Soon, the search algorithm will be favoring other paths even if the h in there are not negative.

The true reason: h'' violate a property of admissible heuristic. Since h is admissible, we have $h(G) = 0$. If $X_0 = G$, we have a negative heuristic value at $h''(G)$, and it is no longer admissible. If $X_0 \neq G$, then it is indeed that the optimality holds - the only change is that more nodes will be likely to be expanded for h' and h'' compared to h .

Q2. Iterative Deepening Search

Pacman is performing search in a maze again! The search graph has a branching factor of b , a solution of depth d , a maximum depth of m , and edge costs that may not be integers. Although he knows breadth first search returns the solution with the smallest depth, it takes up too much space, so he decides to try using iterative deepening. As a reminder, in standard depth-first iterative deepening we start by performing a depth first search terminated at a maximum depth of one. If no solution is found, we start over and perform a depth first search to depth two and so on. This way we obtain the shallowest solution, but use only $O(bd)$ space.

But Pacman decides to use a variant of iterative deepening called **iterative deepening A***, where instead of limiting the depth-first search by depth as in standard iterative deepening search, we can limit the depth-first search by the f value as defined in A* search. As a reminder $f[node] = g[node] + h[node]$ where $g[node]$ is the cost of the path from the start state and $h[node]$ is a heuristic value estimating the cost to the closest goal state.

In this question, all searches are tree searches and **not** graph searches.

- (a) Complete the pseudocode outlining how to perform iterative deepening A* by choosing the option from the next page that fills in each of these blanks. Iterative deepening A* should return the solution with the lowest cost when given a consistent heuristic. (Note: Consistency is out of scope in this semester, but for the sake of this problem, remember that a consistent heuristic is one that makes sure you never have to revisit the same node multiple times in A* search.) Note that cutoff is a boolean and new-limit is a number.

```
function ITERATIVE-DEEPENING-TREE-SEARCH(problem)
  start-node ← MAKE-NODE(INITIAL-STATE[problem])
  limit ←  $f[start-node]$ 
  loop
    fringe ← MAKE-STACK(start-node)
    new-limit ← (i)
    cutoff ← (ii)
    while fringe is not empty do
      node ← REMOVE-FRONT(fringe)
      if GOAL-TEST(problem, STATE[node]) then
        return node
      end if
      for child-node in EXPAND(STATE[node], problem) do
        if  $f[child-node] \leq limit$  then
          fringe ← INSERT(child-node, fringe)
          new-limit ← (iii)
          cutoff ← (iv)
        else
          new-limit ← (v)
          cutoff ← (vi)
        end if
      end for
    end while
    if not cutoff then
      return failure
    end if
    limit ← (vii)
  end loop
end function
```

A_1	$-\infty$	A_2	0	A_3	∞	A_4	limit
B_1	True	B_2	False	B_3	cutoff	B_4	not cutoff
C_1	new-limit	C_2	new-limit + 1	C_3	new-limit + f[node]	C_4	new-limit + f[child-node]
C_5	MIN(new-limit, f[node])	C_6	MIN(new-limit, f[child-node])	C_7	MAX(new-limit, f[node])	C_8	MAX(new-limit, f[child-node])

- (i) ☐ A_1 ☐ A_2 ☒ A_3 ☐ A_4
- (ii) ☐ B_1 ☒ B_2 ☐ B_3 ☐ B_4
- (iii) ☒ C_1 ☐ C_2 ☐ C_3 ☐ C_4
☐ C_5 ☐ C_6 ☒ C_7 ☒ C_8
- (iv) ☐ B_1 ☐ B_2 ☒ B_3 ☐ B_4
- (v) ☐ C_1 ☐ C_2 ☐ C_3 ☐ C_4
☐ C_5 ☒ C_6 ☐ C_7 ☐ C_8
- (vi) ☒ B_1 ☐ B_2 ☐ B_3 ☐ B_4
- (vii) ☒ C_1 ☐ C_2 ☐ C_3 ☐ C_4
☐ C_5 ☐ C_6 ☐ C_7 ☐ C_8

The cutoff variable keeps track of whether there are items that aren't being explored because of the limit. If cutoff is false and the algorithm has exited the while, no nodes were cutoff (not added to the fringe because of the limit). This scenario suggests that there is no solution.

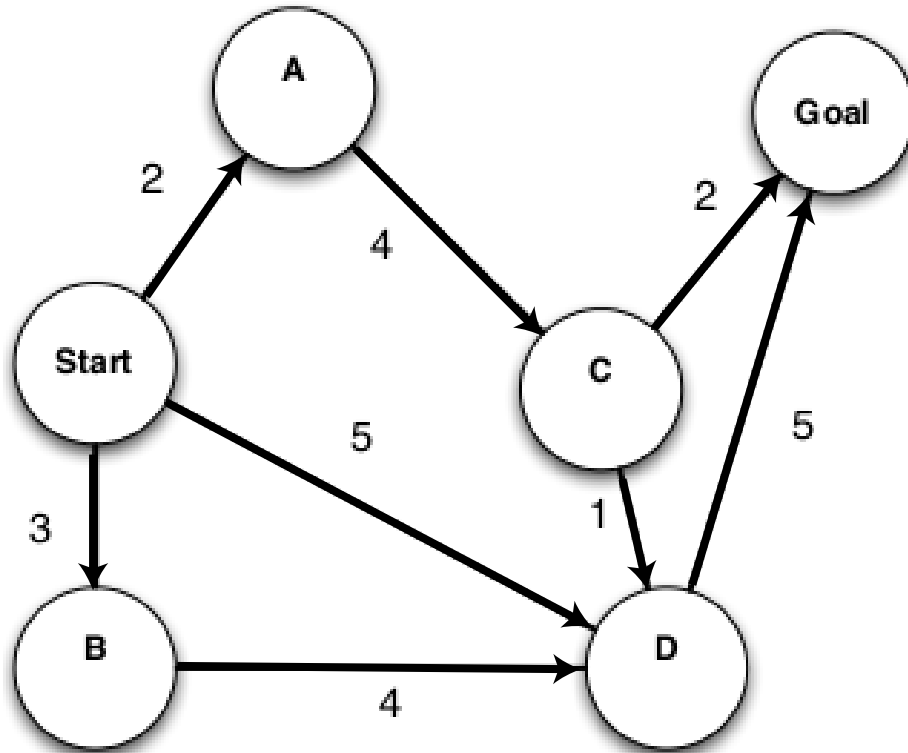
In order to ensure that iterative deepening A* obtains the lowest cost solution efficiently, we want to increase the limit as much as we can while guaranteeing optimality. Setting new-limit to the smallest f cost of nodes that were cutoff achieves this. When nodes aren't cutoff (part iii), the new-limit should not change. Hence C1, C7, C8, or a combination of the three were accepted as answers.

- (b) Assuming there are no ties in f value between nodes, which of the following statements about the number of nodes that iterative deepening A* expands is True? If the same node is expanded multiple times, count all of the times that it is expanded. If none of the options are correct, mark None of the above.

- ☒ The number of times that iterative deepening A* expands a node is greater than or equal to the number of times A* will expand a node.
- ☐ The number of times that iterative deepening A* expands a node is less than or equal to the number of times A* will expand a node.
- ☐ We don't know if the number of times iterative deepening A* expands a node is more or less than the number of times A* will expand a node.
- ☐ None of the above

Iterative deepening A* runs depth first search multiples at different limit values. This causes iterative deepening A* to expand certain nodes multiple times.

1 Search Algorithms in Action



For each of the following graph search strategies, work out the order in which states are expanded, as well as the path returned by graph search. In all cases, assume ties resolve in such a way that states with earlier alphabetical order are expanded first. Remember that in graph search, a state is expanded only once.

a) Depth-first search.

States Expanded: Start, A, C, D, Goal

Path Returned: Start-A-C-D-Goal

b) Breadth-first search.

States Expanded: Start, A, B, D, C, Goal

Path Returned: Start-D-Goal

c) Uniform cost search.

States Expanded: Start, A, B, D, C, Goal

Path Returned: Start-A-C-Goal

2 Search and Heuristics

Imagine a car-like agent wishes to exit a maze like the one shown below:

1. The agent is directional and at all times faces some direction $d \in (N, S, E, W)$.
2. With a single action, the agent can *either* move forward at an adjustable velocity v *or* turn. The turning actions are *left* and *right*, which change the agent's direction by 90 degrees. Turning is only permitted when the velocity is zero (and leaves it at zero).
3. The moving actions are *fast* and *slow*. *Fast* increments the velocity by 1 and *slow* decrements the velocity by 1; in both cases the agent then moves a number of squares equal to its NEW adjusted velocity (see example below).
4. A consequence of this formulation is that it is **impossible** for the agent to move in the same nonzero velocity for two consecutive timesteps.
5. Any action that would result in a collision with a wall or reduce v below 0/above a maximum speed $V_{\max}$ is illegal.
6. The agent's goal is to find a plan which parks it (stationary) on the exit square using as few actions (time steps) as possible.

As an example: if at timestep t the agent's current velocity is 2, by taking the *fast* action, the agent first increases the velocity to 3 and move 3 squares forward, such that at timestep $t + 1$ the agent's current velocity will be 3 and will be 3 squares away from where it was at timestep t . If instead the agent takes the *slow* action, it first decreases its velocity to 1 and then moves 1 square forward, such that at timestep $t + 1$ the agent's current velocity will be 1 and will be 1 squares away from where it was at timestep t . If, with an instantaneous velocity of 1 at timestep $t + 1$, it takes the slow action again, the agent's current velocity will become 0, and it will not move at timestep $t + 1$. Then at timestep $t + 2$, it will be free to turn if it wishes. Note that the agent could not have turned at timestep $t + 1$ when it had a current velocity of 1, because it has to be stationary to turn.

- (a) If the grid is M by N , what is the size of the state space? Justify your answer. You should assume that all configurations are reachable from the start state.

The size of the state space is $4MN(V_{\max} + 1)$. The state representation is (direction facing, x , y , speed). Note that the speed can take any value in $\{0, \dots, V_{\max}\}$.

- (b) Is the Manhattan distance from the agent's location to the exit's location admissible? Why or why not?

No, Manhattan distance is not an admissible heuristic. The agent can move at an average speed of greater than 1 (by first speeding up to V_{max} and then slowing down to 0 as it reaches the goal), and so can reach the goal in less time steps than there are squares between it and the goal. A specific example: At timestep 0, the agent starts stationary at square 0 and the target is 9 squares away at square 9. At timestep 0, the agent takes the *fast* action and ends up at square 1 with a velocity of 1. At timestep 1, the agent takes the *fast* action and ends up at square 3 with a velocity of 2. At timestep 2, the agent takes the *fast* action and ends up at square 6 with a velocity of 3. At timestep 3, the agent takes the *slow* action and ends up at square 8 with a velocity of 2. At timestep 4, the agent takes the *slow* action and ends up at square 9 with a velocity of 1. At timestep 5, the agent takes the *slow* action and stays at square 9 with a velocity of 0. Therefore, the agent can move 9 squares by taking 6 actions.

- (c) State and justify a non-trivial admissible heuristic for this problem which is not the Manhattan distance to the exit.

There are many answers to this question. Here are a few, in order of weakest to strongest:

- (a) The number of turns required for the agent to face the goal.
- (b) Consider a relaxation of the problem where there are no walls, the agent can turn, change speed arbitrarily, and maintain constant velocity. In this relaxed problem, the agent would move with V_{max} , and then suddenly stop at the goal, thus taking $d_{manhattan}/V_{max}$ time.
- (c) We can improve the above relaxation by accounting for the acceleration and deceleration dynamics. In this case the agent will have to accelerate from 0 from the start state, maintain a constant velocity of V_{max} , and slow down to 0 when it is about to reach the goal. Note that this heuristic will always return a greater value than the previous one, but is still not an overestimate of the true cost to reach the goal. We can say that this heuristic *dominates* the previous one.

In particular, let us assume that $d_{manhattan}$ is greater than and equal to the distance it takes to accelerate to and decelerate from V_{max} (In the case that $d_{manhattan}$ is smaller than this distance, we can still use $d_{manhattan}/V_{max}$ as a heuristic). We can break up the $d_{manhattan}$ into three parts: d_{accel} , $d_{V_{max}}$, and d_{decel} . The agent travels a distance of d_{accel} when it accelerates from 0 to V_{max} velocity. The agent travels a distance of d_{decel} when it decelerates from V_{max} to 0 velocity. In between acceleration and deceleration, the agent travels a distance of $d_{V_{max}} = d_{manhattan} - d_{accel} - d_{decel}$. $d_{accel} = 1 + 2 + 3 + \dots + V_{max} = \frac{(V_{max})(V_{max}+1)}{2}$ and $d_{decel} = (V_{max} - 1) + (V_{max} - 2) + \dots + 1 + 0 = \frac{(V_{max})(V_{max}-1)}{2}$. So $d_{V_{max}} = d_{manhattan} - \frac{(V_{max})(V_{max}+1)}{2} - \frac{(V_{max})(V_{max}-1)}{2} = d_{manhattan} - V_{max}^2$. It takes V_{max} steps to travel the initial d_{accel} , $\frac{d_{manhattan} - V_{max}^2}{V_{max}}$ steps to travel the middle $d_{V_{max}}$ and V_{max} steps to travel the last d_{decel} . Therefore, our heuristic is

$$\begin{cases} \frac{d_{manhattan}}{V_{max}}, & \text{if } d_{manhattan} \leq V_{max}^2 = d_{accel} + d_{decel} \\ \frac{d_{manhattan}}{V_{max}} + V_{max}, & \text{if } d_{manhattan} > V_{max}^2 = d_{accel} + d_{decel} \end{cases}$$

- (d) If we used an inadmissible heuristic in A* graph search, would the search be complete? Would it be optimal?

If the heuristic function is bounded, then A* graph search would visit all the nodes eventually, and would find a path to the goal state if there exists one. An inadmissible heuristic does not guarantee optimality as it can make the good optimal goal look as though it is very far off, and take you to a suboptimal goal.

- (e) Give a possible advantage that an inadmissible heuristic might have over an admissible one.

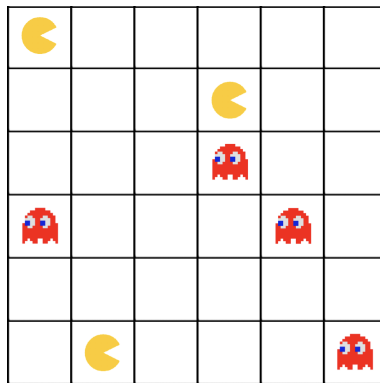
The time to solve an A* search problem is a function of two factors: the number of nodes expanded, and the time spent per node. An inadmissible heuristic may be faster to compute, leading to a solution that is obtained faster due to less time spent per node. It can also be a closer estimate to the actual cost function (even though at times it will overestimate!), thus expanding less nodes. We lose the guarantee of optimality by using an inadmissible heuristic. But sometimes we may be okay with finding a suboptimal solution to a search problem.

Q3. Pacfriends Unite

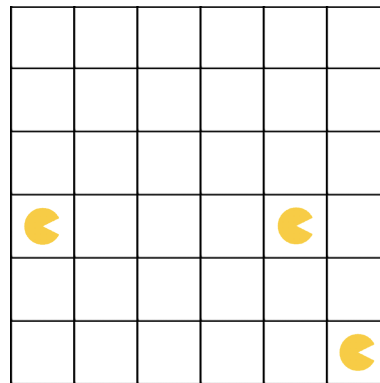
Pacman and his Pacfriends have decided to combine forces and go on the offensive, and are now chasing ghosts instead! In a grid of size M by N , Pacman and $P - 1$ of his Pacfriends are moving around to collectively eliminate **all** of the ghosts in the grid by stepping on the same square as each of them. Moving onto the same square as a ghost will eliminate it from the grid, and move the Pacman into that square.

Every turn, Pacman and his Pacfriends may choose one of the following four actions: *left*, *right*, *up*, *down*, but may not collide with each other. In other words, any action that would result in two or more Pacmen occupying the same square will result in no movement for either Pacman or the Pacfriends. Additionally, Pacman and his Pacfriends are **indistinguishable** from each other. There are a total of G ghosts, which are indistinguishable from each other, and cannot move.

Treating this as a search problem, we consider each configuration of the grid to be a state, and the goal state to be the configuration where **all** of the ghosts have been eliminated from the board. Below is an example starting state, as well as an example goal state:



(a) Possible Start State



(b) Possible Goal State

Assume each of the following subparts are **independent** from each other. **Also assume that regardless of how many Pacmen move in one turn, the total cost of moving is still 1.**

(a) Suppose that Pacman has no Pacfriends :(, so $P = 1$.

(i) What is the size of the minimal state space representation given this condition? Recall that $P = 1$.

- | | |
|------------------------------------|--|
| ☐ MN | ☐ 2^{MN} |
| ☐ MNG | ☐ 2^{MN+G} |
| ☐ $(MN)^G$ | ☐ $G(2)^{MN}$ |
| ☐ $(MN)^{G+1}$ | ☒ $MN(2)^G$ |

Since $P = 1$, we only need to keep track of the position of Pacman, as well as whether each of the G ghosts has been eliminated. We can keep track of this with MN and 2^G respectively; therefore, the size of the minimal representation is the product of the two, which is $MN(2)^G$. Note that we do not need to keep track of the ghosts' positions since they are stationary.

For each of the following heuristics, select whether the heuristic is admissible or inadmissible. Recall that $P = 1$.

(ii) $h(n)$ = the sum of the Manhattan distances from Pacman to every ghost.

☐ admissible

☒ inadmissible

In this case, we can consider a dummy example, such that all the ghosts are adjacent and lined up horizontally, and Pacman is one square to their left. If there are three ghosts, then the sum of their Manhattan distances from Pacman is $1 + 2 + 3 = 6$, which is an overestimate of the actual cost 3, since the Pacman only needs to move three spaces to actually eliminate all the ghosts.

- (iii) $h(n)$ = the number of ghosts times the maximum Manhattan distance between Pacman and any of the ghosts.

☐ admissible

☒ inadmissible

Considering the same example as in the previous part, this heuristic also overestimates the cost, since the number of ghosts $\times$ Manhattan distance between Pacman and the furthest ghost is $3 \times 3 = 9 > 3$. This question was randomized to say 'minimum Manhattan distance' as well, but the answer is still neither. Consider the case where Pacman is very far from the closest ghost, but all the ghosts are next to each other. The heuristic will give us an overestimate to collect all the ghosts, because going the minimum distance value to each ghost is an overestimate once you've reached the closest ghost.

- (iv) $h(n)$ = the number of remaining ghosts.

☒ admissible

☐ inadmissible

Without his Pacfriends, Pacman can only eliminate at most one ghost each turn, and therefore the number of ghosts will be a lower bound on the number of moves necessary to eliminate all the ghosts, making this heuristic admissible. (Note: Consistency is not tested this semester, but it might be interesting to know.) Additionally, since between moves, the number of ghosts can only be reduced by at most one, the heuristic is also consistent.

- (b) Suppose that Pacman has exactly one less Pacfriend than there are number of ghosts :) ; therefore $P = G$. Recall that Pacman and his Pacfriends are indistinguishable from each other.

- (i) What is the size of the minimal state space representation given this condition? Recall that $P = G$.

☐ MNP

☐ $(MN)^G P$

☐ $\binom{MN}{P} (MN)^G$

☐ $MNGP$

☐ $(MN)^{G+1}$

☐ $\binom{MN}{P} \binom{MN}{G}$

☐ $(MN)^G$

☐ $(MN)^{(G+1)P}$

☐ 2^{MN}

☐ $(MN)^{(G+P)}$

☐ $\binom{MN}{P}$

☐ 2^{MN+G+P}

☐ $(MN)^P 2^G$

☒ $\binom{MN}{P} 2^G$

☐ $GP(2)^{MN}$

Since P can be any number of Pacman and Pacfriends, arbitrarily equal to G , we need to keep track of the position of all the Pacmen, as well as whether each of the G ghosts has been eliminated; however, since the Pacmen are **indistinguishable** from each other, we can consider states where different Pacmen have swapped positions to be the same. Therefore, we can keep track of unique Pacmen configurations using $\binom{MN}{P}$, which is the number of ways to select P Pacman positions from a grid of M by N . Therefore, the size of the minimal representation is the product of this and 2^G , which is $\binom{MN}{P} 2^G$.

For each of the following heuristics, select whether the heuristic is admissible or inadmissible. Recall that $P = G$.

- (ii) $h(n)$ = the largest of the Manhattan distances between each Pacman and its closest ghost.

☐ admissible

☒ inadmissible

Consider the case where we have two Pacmen and two ghosts, and one Pacman is very close to both ghosts, while the other is very far away. This heuristic would overestimate the cost since it would take the distance from the furthest Pacman to the ghosts as its metric, when the closer Pacman could just eliminate both ghosts quicker.

(iii) $h(n)$ = the smallest of the Manhattan distances between each Pacman and its closest ghost.

☒ admissible

☐ inadmissible

At minimum, it will take the smallest of the distances between each Pacman and its closest ghost for any of the ghosts to be eliminated. Therefore this heuristic will always underestimate the true cost and is therefore admissible.

(iv) $h(n)$ = the number of remaining ghosts.

☐ admissible

☒ inadmissible

Since multiple ghosts could be eliminated in one move because there is more than one Pacman eliminating them, this heuristic can overestimate the cost to the goal state, and is therefore inadmissible.

(v) $h(n) = \frac{\text{number of remaining ghosts}}{P}$.

☒ admissible

☐ inadmissible

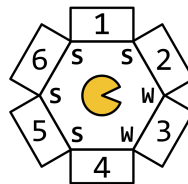
Since we are given that $P = G$, the maximum value of this heuristic is 1, since $h_6(n)$ is at most G . Therefore this heuristic is admissible since it has a value always less than or equal to 1, the cost of taking a turn, and when there are no remaining ghosts left, the heuristic evaluates to 0, so it will underestimate the cost to the goal state.

1 CSPs: Trapped Pacman

Pacman is trapped! He is surrounded by mysterious corridors, each of which leads to either a pit (P), a ghost (G), or an exit (E). In order to escape, he needs to figure out which corridors, if any, lead to an exit and freedom, rather than the certain doom of a pit or a ghost.

The one sign of what lies behind the corridors is the wind: a pit produces a strong breeze (S) and an exit produces a weak breeze (W), while a ghost doesn't produce any breeze at all. Unfortunately, Pacman cannot measure the strength of the breeze at a specific corridor. Instead, he can stand *between* two adjacent corridors and feel the max of the two breezes. For example, if he stands between a pit and an exit he will sense a strong (S) breeze, while if he stands between an exit and a ghost, he will sense a weak (W) breeze. The measurements for all intersections are shown in the figure below.

Also, while the total number of exits might be zero, one, or more, Pacman knows that two neighboring squares will *not* both be exits.



Pacman models this problem using variables X_i for each corridor i and domains P, G, and E.

- (a) State the binary and/or unary constraints for this CSP (either implicitly or explicitly).

Binary:

$$\begin{aligned} X_1 = P \text{ or } X_2 = P, & \quad X_2 = E \text{ or } X_3 = E, \\ X_3 = E \text{ or } X_4 = E, & \quad X_4 = P \text{ or } X_5 = P, \\ X_5 = P \text{ or } X_6 = P, & \quad X_1 = P \text{ or } X_6 = P, \\ \forall i, j \text{ s.t. Adj}(i, j) & \neg(X_i = E \text{ and } X_j = E) \end{aligned}$$

Unary:

$$\begin{aligned} X_2 &\neq P, \\ X_3 &\neq P, \\ X_4 &\neq P \end{aligned}$$

Note: This is just one of many solutions. The answers below will be based on this formulation.

- (b) Suppose we assign X_1 to E. Perform forward checking after this assignment. Also, enforce unary constraints.
- (c) Suppose forward checking returns the following set of possible assignments:

X_1			E
X_2			
X_3		G	E
X_4		G	E
X_5	P	G	E
X_6	P		

X_1	P		
X_2		G	E
X_3		G	E
X_4		G	E
X_5	P		
X_6	P	G	E

According to MRV, which variable or variables could the solver assign first?

X_1 or X_5 (tie breaking)

(d) Assume that Pacman knows that $X_6 = G$. List all the solutions of this CSP or *none* if no solutions exist.

(P,E,G,E,P,G)

(P,G,E,G,P,G)

2 Course Scheduling

You are in charge of scheduling for computer science classes that meet Mondays, Wednesdays and Fridays. There are 5 classes that meet on these days and 3 professors who will be teaching these classes. You are constrained by the fact that each professor can only teach one class at a time.

The classes are:

1. Class 1 - Intro to Programming: meets from 8:00-9:00am
2. Class 2 - Intro to Artificial Intelligence: meets from 8:30-9:30am
3. Class 3 - Natural Language Processing: meets from 9:00-10:00am
4. Class 4 - Computer Vision: meets from 9:00-10:00am
5. Class 5 - Machine Learning: meets from 10:30-11:30am

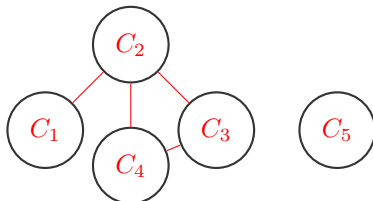
The professors are:

1. Professor A, who is qualified to teach Classes 1, 2, and 5.
2. Professor B, who is qualified to teach Classes 3, 4, and 5.
3. Professor C, who is qualified to teach Classes 1, 3, and 4.

1. Formulate this problem as a CSP problem in which there is one variable per class, stating the domains (after enforcing unary constraints), and binary constraints. Constraints should be specified formally and precisely, but may be implicit rather than explicit.

Variables	Domains (or unary constraints)	Binary Constraints
C_1	$\{A, C\}$	$C_1 \neq C_2$
C_2	$\{A\}$	$C_2 \neq C_3$
C_3	$\{B, C\}$	$C_2 \neq C_4$
C_4	$\{B, C\}$	$C_3 \neq C_4$
C_5	$\{A, B\}$	

2. Draw the constraint graph associated with your CSP.



Q1. Games

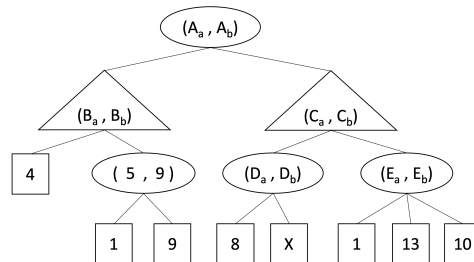
Alice is playing a two-player game with Bob, in which they move alternately. Alice is a maximizer. Although Bob is also a maximizer, Alice believes Bob is a minimizer with probability 0.5, and a maximizer with probability 0.5. Bob is aware of Alice's assumption.

In the game tree below, square nodes are the outcomes, triangular nodes are Alice's moves, and round nodes are Bob's moves. Each node for Alice/Bob contains a tuple, the left value being Alice's expectation of the outcome, and the right value being Bob's expectation of the outcome.

Tie-breaking: choose the left branch.

The left values are Alice's expectations, and are the only thing Alice can refer to when making decisions.

The right values are Bob's expectations, and they also accurately track the expected outcome of the game according to each choice of branching (regardless of it is Alice's or Bob's decision, since Bob has all the information). Hence the right values are accurate information about the game, and would be what Bob looks at when making his decisions. However, when it is Alice's turn to make decisions, Bob will think about how Alice would maximize the outcome w.r.t to what she believes, and he will update his expectations accordingly.



- (a) In the blanks below, fill in the tuple values for tuples (B_a, B_b) and (E_a, E_b) from the above game tree.

$(B_a, B_b) = ($ 5 $,$ 9 $)$

$(E_a, E_b) = ($ 7 $,$ 13 $)$

For a square node, its value v means the same to Alice and Bob, i.e., we can think of it as a tuple (v, v) .

The left value of Alice's nodes is the maximum of the left values of its children nodes, since Alice believes that the values of the nodes are given by left values, and it's her turn of action, so she will choose the largest value.

The right value of Alice's nodes is the right value from the child node that attains the maximum left value since Bob's expectation is consistent with how Alice will act.

So for a triangular node, its tuple is the same as its child that has the maximum left value.

The left value of Bob's nodes is the average of the maximum and minimum of the left values of its children nodes since Alice believes Bob is 50% possible to be adversarial and 50% possible to be friendly.

The right value of Bob's nodes is the maximum of the right values of the immediate children nodes since Bob would choose the branch that gives the maximum outcome during his turn.

So for a round node, $\text{left} = 0.5(\max(\text{children.left}) + \min(\text{children.left}))$, and $\text{right} = \max(\text{children.right})$.

- (b) In this part, we will determine the values for tuple (D_a, D_b) .

(i) $D_a =$ ☐ 8 ☐ X ☐ $8+X$ ☒ $4+0.5X$ ☐ $\min(8, X)$ ☐ $\max(8, X)$

(ii) $D_b =$ ☐ 8 ☐ X ☐ $8+X$ ☐ $4+0.5X$ ☐ $\min(8, X)$ ☒ $\max(8, X)$

It's a round node, so $\text{left} = 0.5(\max(\text{children.left}) + \min(\text{children.left}))$, and $\text{right} = \max(\text{children.right})$.
 Its children: (8,8) and (X,X). So $\text{left} = 0.5(8+X) = 4+0.5X$, and $\text{right} = \max(8, X)$.

- (c) Fill in the values for tuple (C_a, C_b) below. For the bounds of X, you may write scalars, ∞ or $-\infty$.
 If your answer contains a fraction, please write down the corresponding **simplified decimal value** in its place. (i.e., 4 instead of $\frac{8}{2}$, and 0.5 instead of $\frac{1}{2}$).

$$1. \text{ If } -\infty < X < \boxed{6}, (C_a, C_b) = (\boxed{7}, \boxed{13})$$

$$2. \text{ Else, } (C_a, C_b) = (\boxed{4+0.5X}, \max(\boxed{8}, \boxed{X}))$$

It's a triangular node, so its tuple is the same as its child that has the maximum left value.
 Its children: $(4+0.5X, \max(8,X))$ and $(7, 13)$.
 So if $4+0.5X < 7$, i.e. $-\infty < X < 6$, it's the same as child node $(7, 13)$, and otherwise it's $(4+0.5X, \max(8,X))$.

- (d) Fill in the values for tuple (A_a, A_b) below. For the bounds of X, you may write scalars, ∞ or $-\infty$.
 If your answer contains a fraction, please write down the corresponding **simplified decimal value** in its place. (i.e., 4 instead of $\frac{8}{2}$, and 0.5 instead of $\frac{1}{2}$).

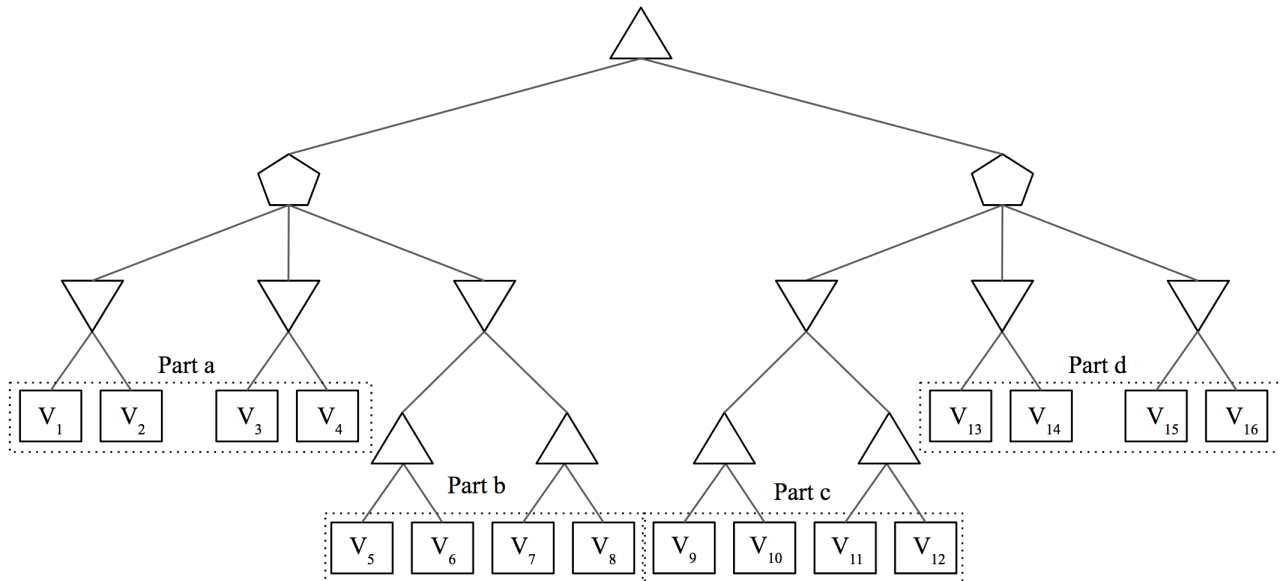
$$1. \text{ If } -\infty < X < \boxed{6}, (A_a, A_b) = (\boxed{6}, \boxed{13})$$

$$2. \text{ Else, } (A_a, A_b) = (\boxed{4.5+0.25X}, \max(\boxed{9}, \boxed{X}))$$

It's a round node, so $\text{left} = 0.5(\max(\text{children.left}) + \min(\text{children.left}))$, and $\text{right} = \max(\text{children.right})$.
 Its children: (5,9) and node "Part (c)".
 If $-\infty < X < 6$, these children are (5,9) and $(7, 13)$.
 $\text{left} = 0.5(\max(\text{children.left}) + \min(\text{children.left})) = 0.5(5+7) = 6$
 $\text{right} = \max(\text{children.right}) = \max(9, 13) = 13$.
 Otherwise $(6 < X < +\infty)$, these children are (5,9) and $(4+0.5X, \max(8,X))$.
 $\text{left} = 0.5(\max(\text{children.left}) + \min(\text{children.left})) = 0.5(5+4+0.5X) = 4.5 + 0.25X$
 $\text{right} = \max(\text{children.right}) = \max(9, \max(8,X)) = \max(9,X)$.

Q2. MedianMiniMax

You're living in utopia! Despite living in utopia, you still believe that you need to maximize your utility in life, other people want to minimize your utility, and the world is a 0 sum game. But because you live in utopia, a benevolent social planner occasionally steps in and chooses an option that is a compromise. Essentially, the social planner (represented as the pentagon) is a median node that chooses the successor with median utility. Your struggle with your fellow citizens can be modelled as follows:



There are some nodes that we are sometimes able to prune. In each part, mark all of the terminal nodes such that **there exists a possible situation** for which the node **can be pruned**. In other words, you must consider **all** possible pruning situations. Assume that evaluation order is **left to right** and all V_i 's are **distinct**.

Note that as long as there exists ANY pruning situation (does not have to be the same situation for every node), you should mark the node as prunable. Also, alpha-beta pruning does not apply here, simply prune a sub-tree when you can reason that its value will not affect your final utility.

- (a) ☐ V_1
☐ V_2
☐ V_3
☐ V_4
☒ None

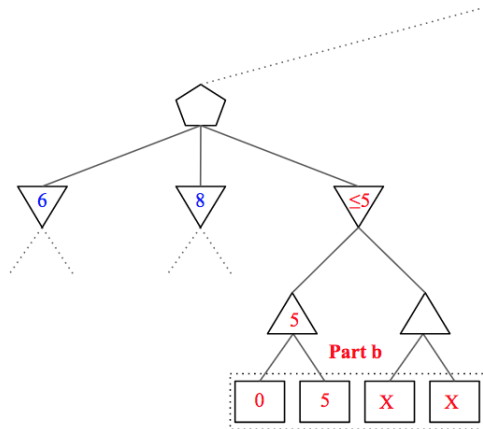
- (b) ☐ V_5
☒ V_6
☒ V_7
☒ V_8
☐ None

- (c) ☐ V_9
☐ V_{10}
☒ V_{11}
☒ V_{12}
☐ None

- (d) ☐ V_{13}
☒ V_{14}
☒ V_{15}
☒ V_{16}
☐ None

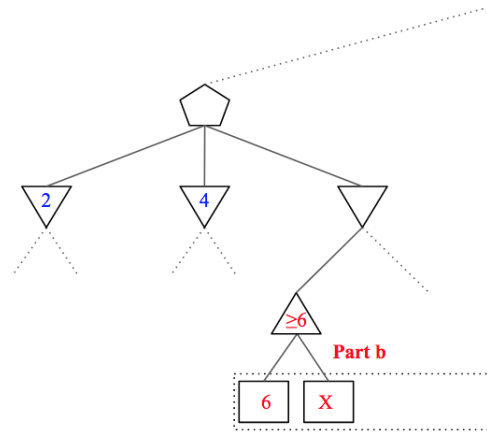
Part a:

For the left median node with three children, at least two of the childrens' values must be known since one of them will be guaranteed to be the value of the median node passed up to the final maximizer. For this reason, none of the nodes in part a can be pruned.



The value of this subtree will only get smaller.

The median node will **NOT** choose the value of this subtree. 6 is the median.



The value of this subtree will only get bigger.

If the value of this subtree is chosen by the minimizer*, it will **NOT** be chosen by the median node.

*It is possible that the median is the value of the subtree to the right that we haven't looked at yet

Part b (pruning V_7, V_8):

Let min_1, min_2, min_3 be the values of the three minimizer nodes in this subtree.

In this case, we may not need to know the final value min_3 . The reason for this is that we may be able to put a bound on its value after exploring only partially, and determine the value of the median node as either min_1 or min_2 if $min_3 \leq \min(min_1, min_2)$ or $min_3 \geq \max(min_1, min_2)$.

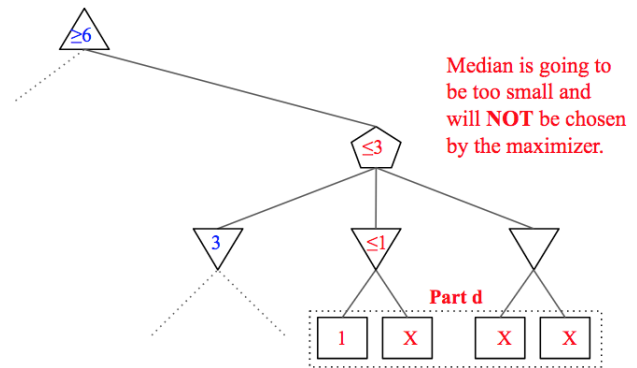
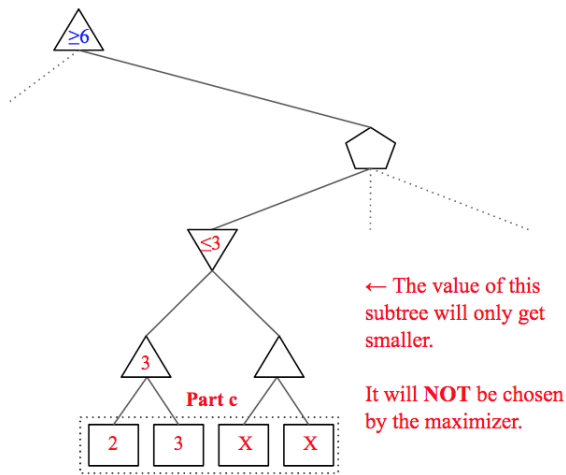
We can put an upper bound on min_3 by exploring the left subtree V_5, V_6 and if $\max(V_5, V_6)$ is lower than both min_1 and min_2 , the median node's value is set as the smaller of min_1, min_2 and we don't have to explore V_7, V_8 in Figure 1.

Part b (pruning V_6):

It's possible for us to put a lower bound on min_3 . If V_5 is larger than both min_1 and min_2 , we do not need to explore V_6 .

The reason for this is subtle, but if the minimizer chooses the left subtree, we know that $min_3 \geq V_5 \geq \max(min_1, min_2)$ and we don't need V_6 to get the correct value for the median node which will be the larger of min_1, min_2 .

If the minimizer chooses the value of the right subtree, the value at V_6 is unnecessary again since the minimizer never chose its subtree.



Part c (pruning V_{11}, V_{12}):

Assume the highest maximizer node has a current value $max_1 \geq Z$ set by the left subtree and the three minimizers on this right subtree have value min_1, min_2, min_3 .

In this part, if $min_1 \leq \max(V_9, V_{10}) \leq Z$, we do not have to explore V_{11}, V_{12} . Once again, the reasoning is subtle, but we can now realize if either $min_2 \leq Z$ or $min_3 \leq Z$ then the value of the right median node is for sure $\leq Z$ and is useless.

Only if both $min_2, min_3 \geq Z$ will the whole right subtree have an effect on the highest maximizer, but in this case the exact value of min_1 is not needed, just the information that it is $\leq Z$. Clearly in both cases, V_{11}, V_{12} are not needed since an exact value of min_1 is not needed.

We will also take the time to note that if $V_9 \geq Z$ we do have to continue the exploring as V_{10} could be even greater and the final value of the top maximizer, so V_{10} can't really be pruned.

Part d (pruning V_{14}, V_{15}, V_{16}):

Continuing from part c, if we find that $min_1 \leq Z$ and $min_2 \leq Z$ we can stop.

We can realize this as soon we explore V_{13} . Once we figure this out, we know that our median node's value must be one of these two values, and neither will replace Z so we can stop.

Q1. Search

For this problem, assume that all of our search algorithms use tree search, unless specified otherwise.

- (a) For each algorithm below, indicate whether the path returned after the modification to the search tree is guaranteed to be identical to the unmodified algorithm. Assume all edge weights are non-negative before modifications.

- (i) Adding additional cost $c > 0$ to every edge weight.

	Yes	No
BFS	☒	☐
DFS	☒	☐
UCS	☐	☒

- (ii) Multiplying a constant $w > 0$ to every edge weight.

	Yes	No
BFS	☒	☐
DFS	☒	☐
UCS	☒	☐

- (b) For part (b), two search algorithms are defined to be **equivalent** if and only if they expand the same states in the same order and return the same path. **Assume all graphs are directed and acyclic.**

- (i) Assume we have access to costs c_{ij} that make running UCS algorithm with these costs c_{ij} equivalent to running BFS. How can we construct new costs c'_{ij} such that running UCS with these costs is equivalent to running DFS?

- ☐ $c'_{ij} = 0$
☐ $c'_{ij} = 1$
☐ $c'_{ij} = c_{ij}$
☒ $c'_{ij} = -c_{ij}$
☐ $c'_{ij} = c_{ij} + \alpha$
☐ Not possible

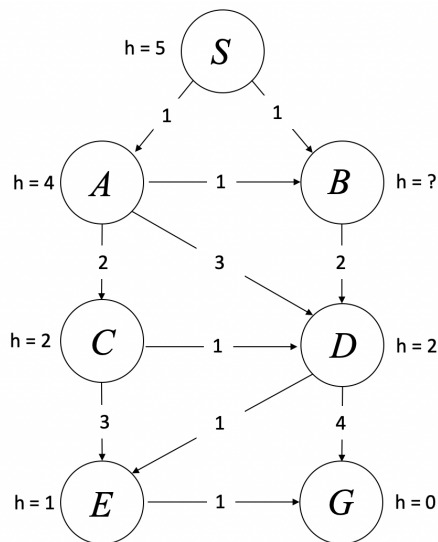
Breadth-First Search expands the node at the shallowest depth first. Assigning a constant positive weight to all edges allows to weigh the nodes by their depth in the search tree. Depth-First Search expands the nodes which were most recently added to the fringe first. Assigning a constant negative weight to all edges essentially allows to reduce the value of the most recently nodes by that constant, making them the nodes with the minimum value in the fringe when using uniform cost search. Hence, we can construct new costs c'_{ij} by flipping the sign of the original costs c_{ij} .

- (ii) Given edge weight $c_{ij} = h(j) - h(i)$, where $h(n)$ is the value of the heuristic function at node n , running UCS on this graph is equivalent to running which of the following algorithm on the same graph?

- ☐ DFS
 ☐ BFS
 ☐ Iterative Deepening
☒ Greedy
 ☐ A*
 ☐ None of the above.

Greedy Search expands the node with the lowest heuristic function value $h(n)$. If $c_{ij} = h(j) - h(i)$, then the cost of a node n on the fringe when running uniform-cost search will be $\sum_{ij} c_{ij} = h(1) - h(start) + h(2) - h(1) + \dots + h(n) - h(n-1) = h(n) - h(start)$. As $h(start)$ is a common constant subtracted from the cost of all nodes on the fringe, the relative ordering of the nodes on the fringe is still determined by $h(n)$, i.e. their heuristic values.

(c) Consider the following graph. $h(n)$ denotes the heuristic function evaluated at node n .



- (i) Given that G is the goal node, and heuristic values are fixed for all nodes other than B , for which values of $h(B)$ will A* tree search be guaranteed to return the optimal path? Fill in the lower and upper bounds or select "impossible."

0 $\leq h(B) \leq$ 4 ☐ Impossible

An admissible heuristic is sufficient for A* tree search to be optimal. Recall the constraint for an admissible heuristic: $\forall n, 0 \leq h(n) \leq h^*(n)$. Since all other nodes satisfy the constraint above, we just need to enforce the constraint on node B, so that $0 \leq h(B) \leq h^*(B) = 4$.

- (ii) With the heuristic values fixed for all nodes other than B , for which values of $h(B)$ will A* graph search be guaranteed to return the optimal path? Either fill in the lower and upper bound or select "impossible."

4 $\leq h(B) \leq$ 4 ☐ Impossible

We need a consistent heuristic for A* tree search to be optimal. Recall the constraint for a consistent heuristic: $\forall A, C \quad h(A) - h(C) \leq \text{cost}(A, C)$. Since all other nodes satisfy the constraint above, we just need to enforce the constraint on node B. Enforcing consistency along $S - B$, gives $h(B) \geq 4$, $A - B$ gives $h(B) \geq 3$, $B - D$ gives $h(B) \leq 4$.

Q2. SpongeBob and Pacman (Search Formulation)

Recall that in Midterm 1, Pacman bought a car, was speeding in Pac-City, and the SpongeBob wasn't able to catch him. Now Pacman has run out of gas, his car has stopped, and he is currently hiding out at an undisclosed location.

In this problem, you are on the SpongeBob side, tryin' to catch Pacman!

There are still p SpongeBob cars in the Pac-city of dimension m by n . In this problem, **all SpongeBob cars can move, with two distinct integer controls: throttle and steering, but Pacman has to stay stationary**. Once one SpongeBob car takes an action which lands him in the same grid as Pacman, Pacman will be arrested and the game ends.

Throttle: $t_i \in \{1, 0, -1\}$, corresponding to {Gas, Coast, Brake}. This controls the **speed** of the car by determining its acceleration. The integer chosen here will be added to his velocity for the next state. For example, if a SpongeBob car is currently driving at 5 grid/s and chooses Gas (1) he will be traveling at 6 grid/s in the next turn.

Steering: $s_i \in \{1, 0, -1\}$, corresponding to {Turn Left, Go Straight, Turn Right}. This controls the **direction** of the car. For example, if a SpongeBob car is facing North and chooses Turn Left, it will be facing West in the next turn.

- (a) Suppose you can **only control 1 SpongeBob car**, and have absolutely no information about the remainder of $p - 1$ SpongeBob cars, or where Pacman stopped to hide. Also, the SpongeBob cars can travel up to 6 grid/s so $0 \leq v \leq 6$ at all times.

- (i) What is the **tightest upper bound** on the size of state space, if your goal is to use search to plan a sequence of actions that guarantees Pacman is caught, no matter where Pacman is hiding, or what actions other SpongeBob cars take. Please note that your state space representation must be able to represent **all** states in the search space.

$$28mn * 2^{mn}$$

There are mn positions in total. At each legal position, there are 7 possible speeds (0, 1, 2, 3, 4, 5, 6), so a factor of 7 is multiplied. In addition, since change of direction depends on orientation of the car, another factor of 4 is multiplied.

The only sequence of actions which guarantees that Pacman is caught is a sequence of actions which visits every location. Thus, we also need to a list of $m * n$ boolean to keep track of whether we have visited a specific grid location, and that is another factor of 2^{mn}

- (ii) What is the maximum branching factor? Your answer may contain integers, m, n .

$$9$$

3 possible throttle inputs, and 3 possible steering inputs. The list of boolean does not affect the branching factor.

- (iii) Which algorithm(s) is/are guaranteed to return a path passing through all grid locations on the grid, if one exists?



Depth First Tree Search



Breadth First Tree Search



Depth First Graph Search



Breadth First Graph Search

Please note the list of boolean is in the state space representation, so we can revisit the same grid position if we have to.

- (iv) Is Breadth First Graph Search guaranteed to return the path with the shortest number of **time steps**, if one exists?



Yes



No

The Breadth First Graph Search is guranteed to return the path with the shortest amount of time, because each edge here represent moving for 1 unit of time.

- (b) Now let's suppose you can control **all** p SpongeBob cars at the same time (and know all their locations), but you still have no information about where Pacman stopped to hide

- (i) Now, you still want to search a sequence of actions such that the paths of p SpongeBob car combined **pass through all $m * n$ grid locations**. Suppose the size of the state space in part (a) was N_1 , and the size of the state space in this part is N_p . Please select the correct relationship between N_p and N_1

☐ $N_p = p * N_1$
☐ $N_p = p^{N_1}$
☐ $N_p = (N_1)^p$
☒ None of the above

In this question, we only need one boolean list of size mn to keep track of whether we have visited a specific grid location. So the size of the state space is bounded by $N_p = (28mn)^p 2^{mn}$, which is none of the above.

- (ii) Suppose the maximum branching factor in part (a) was b_1 , and the maximum branching factor in this part is b_p . Please select the correct relationship between b_p and b_1

☐ $b_p = p * b_1$
☐ $b_p = p^{b_1}$
☒ $b_p = (b_1)^p$
☐ None of the above

For example, the case of $p = 2$ means two cars can do all 9 options, so the branching factor is $9^2 = 81$. In general, the branching factor is then b_1^p .

Introduction to Artificial Intelligence Exam Prep 2 Solutions

Q1. CSPs: Potluck Pandemonium

The potluck is coming up and the staff haven't figured out what to bring yet! They've pooled their resources and determined that they can bring some subset of the following items.

1. Pho
2. Apricots
3. Frozen Yogurt
4. Fried Rice
5. Apple Pie
6. Animal Crackers

There are five people on the course staff: Taylor, Jonathan, Faraz, Brian, and Alvin. Each of them will only bring one item to the potluck.

- i. If (F)araz brings the same item as someone else, it cannot be (B)rian.
- ii. (A)lvin has pho-phobia so he won't bring Pho, but he'll be okay if someone else brings it.
- iii. (B)rian is no longer allowed near a stove, so he can only bring items 2, 3, or 6.
- iv. (F)araz literally can't even; he won't bring items 2, 4, or 6.
- v. (J)onathan was busy, so he didn't see the last third of the list. Therefore, he will only bring item 1, 2, 3, or 4.
- vi. (T)aylor will only bring an item that is before an item that (J)onathan brings.
- vii. (T)aylor is allergic to animal crackers, so he won't bring item 6. (If someone else brings it, he'll just stay away from that table.)
- viii. (F)araz and (J)onathan will only bring items that have the same first letter (e.g. Frozen Yogurt and Fried Rice).
- ix. (B)rian will only bring an item that is after an item that (A)lvin brings on the list.
- x. (J)onathan and (T)aylor want to be unique; they won't bring the same item as anyone else.

This page is repeated as the second-to-last page of this midterm for you to rip out and use for reference as you work through the problem.

(a) Which of the listed constraints are unary constraints?

i ☐

ii ☒

iii ☒

iv ☒

v ☒

vi ☐

vii ☒

viii ☐

ix ☐

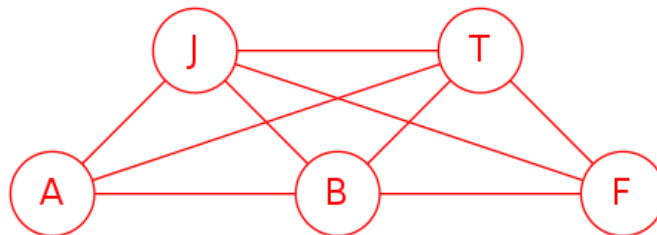
x ☐

(b) Rewrite implicit constraint viii. as an explicit constraint.

$(F, J) \in \{ (3, 4), (4, 3), (2, 5), (5, 2), (2, 6), (6, 2), (5, 6), (6, 5), (1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (6, 6) \}$

(c) How many edges are there in the constraint graph for this CSP?

There are 9 edges in this constraint graph.



(d) The table below shows the variable domains after all unary constraints have been enforced.

A		2	3	4	5	6
B		2	3			6
F	1		3		5	
J	1	2	3	4		
T	1	2	3	4	5	

Following the Minimum Remaining Values heuristic, which variable should we assign first? Break all ties alphabetically.

A ☐

B ☒

F ☐

J ☐

T ☐

- (e) To decouple this from the previous question, assume that we choose to assign (F)araz first. In this question, we will choose which value to assign to using the Least Constraining Value method.

To determine the number of remaining values, enforce arc consistency to prune the domains. Then, count the total number of possible assignments (**not** the total number of remaining values). It may help you to enforce arc consistency twice, once before assigning values to (F)araz, and then again after assigning a value.

The domains after enforcing unary constraints are reproduced in each subquestion. The grids are provided as scratch space and **will not** be graded. Only numbers written in the blanks will be graded. The second grid is provided as a back-up in case you mess up on the first one. More grids are also provided on the second-to-last page of the exam.

- (i) Assigning $F = 1$ results in 0 possible assignments.

A		2	3	4	5	6
B		2	3			6
F	1		3		5	
J	1	2	3	4		
T	1	2	3	4	5	

A		2	3	4	5	6
B		2	3			6
F	1		3		5	
J	1	2	3	4		
T	1	2	3	4	5	

Assigning $F = 1$ leaves no possible values in J's domain (due to constraint viii).

- (ii) Assigning $F = 3$ results in 5 possible assignments.

A		2	3	4	5	6
B		2	3			6
F	1		3		5	
J	1	2	3	4		
T	1	2	3	4	5	

A		2	3	4	5	6
B		2	3			6
F	1		3		5	
J	1	2	3	4		
T	1	2	3	4	5	

Assigning $F = 3$ leaves J's domain as {4}. Enforcing arc consistency gives $A = \{2, 3, 5\}$, $B = \{6\}$, and $T = \{1, 2\}$. Therefore, the 5 possible assignments are $(A, B, F, J, T) = (2, 6, 3, 4, 1), (3, 6, 3, 4, 1), (5, 6, 3, 4, 1), (3, 6, 3, 4, 2), (5, 6, 3, 4, 2)$.

- (iii) Assigning $F = 5$ results in 3 possible assignments.

A		2	3	4	5	6
B		2	3			6
F	1		3		5	
J	1	2	3	4		
T	1	2	3	4	5	

A		2	3	4	5	6
B		2	3			6
F	1		3		5	
J	1	2	3	4		
T	1	2	3	4	5	

Assigning $F = 5$ leaves J's domain as {2}. Enforcing arc consistency gives $A = \{3, 4, 5\}$, $B = \{6\}$, and $T = \{1\}$. Therefore, the 3 possible assignments are $(A, B, F, J, T) = (3, 6, 5, 2, 1), (4, 6, 5, 2, 1), (5, 6, 5, 2, 1)$.

- (iv) Using the LCV method, which value should we assign to F? If there is a tie, choose the lower number. (e.g. If both 1 and 2 have the same value, then fill 1.)

1 ☐ 2 ☐ 3 ☒ 4 ☐ 5 ☐ 6 ☐

Q2. CSPs

In this question, you are trying to find a four-digit number satisfying the following conditions:

1. the number is odd,
2. the number only contains the digits 1, 2, 3, 4, and 5,
3. each digit (except the leftmost) is strictly larger than the digit to its left.

(a) CSPs

We will model this as a CSP where the variables are the four digits of our number, and the domains are the five digits we can choose from. The last variable only has 1, 3, and 5 in its domain since the number must be odd. The constraints are defined to reflect the third condition above. Thus before we start executing any algorithms, the domains are

1 2 3 4 5	1 2 3 4 5	1 2 3 4 5	1 2 3 4 5
-----------	-----------	-----------	-----------

- (i) Before assigning anything, enforce arc consistency. Write the values remaining in the domain of each variable after arc consistency is enforced.

1 2 3 4 5	1 2 3 4 5	1 2 3 4 5	1 2 3 4 5
-----------	-----------	-----------	-----------

- (ii) With the domains you wrote in the previous part, which variable will the MRV (Minimum Remaining Value) heuristic choose to assign a value to first? If there is a tie, choose the leftmost variable.

- ☐ The first digit (leftmost)
☐ The second digit
☐ The third digit
☒ The fourth digit (rightmost)

- (iii) Now suppose we assign to the leftmost digit first. Assuming we will continue filtering by enforcing arc consistency, which value will LCV (Least Constraining Value) choose to assign to the leftmost digit? **Break ties from large (5) to small (1).**

- ☒ 1
☐ 2
☐ 3
☐ 4
☐ 5

- (iv) Now suppose we are running min-conflicts to try to solve this CSP. If we start with the number 1332, what will our number be after one iteration of min-conflicts? Break variable selection ties from left to right, and **break value selection ties from small (1) to large (5).**

1232

(b) The following questions are completely unrelated to the above parts. Assume for these following questions, there are only binary constraints unless otherwise specified.

(i) [true or false] When enforcing arc consistency in a CSP, the set of values which remain when the algorithm terminates does not depend on the order in which arcs are processed from the queue.

(ii) [true or false] Once arc consistency is enforced as a pre-processing step, forward checking can be used during backtracking search to maintain arc consistency for all variables.

False. Forward checking makes the current variable arc-consistent, but doesn't look ahead and make all the other variables arc-consistent.

(iii) In a general CSP with n variables, each taking d possible values, what is the worst case time complexity of enforcing arc consistency using the AC-3 method discussed in class?

☐ 0 ☐ $O(1)$ ☐ $O(nd^2)$ ☒ $O(n^2d^3)$ ☐ $O(d^n)$ ☐ ∞

$O(n^2d^3)$. There are up to n^2 constraints. There are d^2 comparisons for enforcing arc consistency per each constraint, and each constraint can be inserted to the queue up to d times because each variable has at most d values to delete.

(iv) In a general CSP with n variables, each taking d possible values, what is the maximum number of times a backtracking search algorithm might have to backtrack (i.e. the number of the times it generates an assignment, partial or complete, that violates the constraints) before finding a solution or concluding that none exists?

☐ 0 ☐ $O(1)$ ☐ $O(nd^2)$ ☐ $O(n^2d^3)$ ☒ $O(d^n)$ ☐ ∞

$O(d^n)$. In general, the search might have to examine all possible assignments.

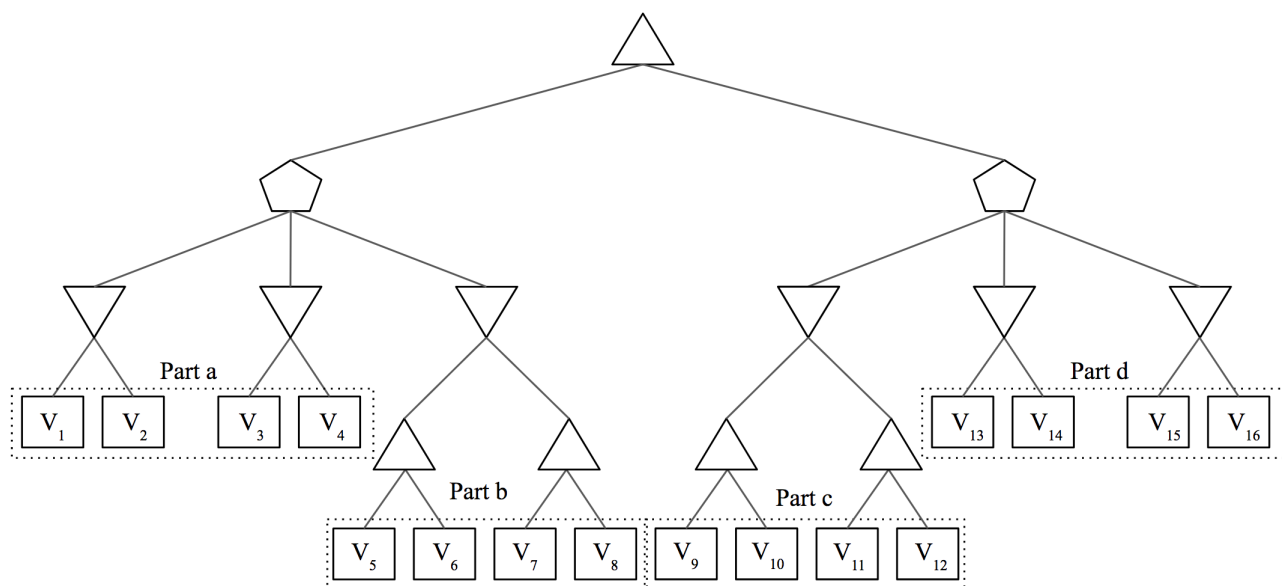
(v) What is the maximum number of times a backtracking search algorithm might have to backtrack in a general CSP, if it is running arc consistency and applying the MRV and LCV heuristics?

☐ 0 ☐ $O(1)$ ☐ $O(nd^2)$ ☐ $O(n^2d^3)$ ☒ $O(d^n)$ ☐ ∞

$O(d^n)$. The MRV and LCV heuristics are often helpful to guide the search, but are not guaranteed to reduce backtracking in the worst case.

Q1. MedianMiniMax

You're living in utopia! Despite living in utopia, you still believe that you need to maximize your utility in life, other people want to minimize your utility, and the world is a 0 sum game. But because you live in utopia, a benevolent social planner occasionally steps in and chooses an option that is a compromise. Essentially, the social planner (represented as the pentagon) is a median node that chooses the successor with median utility. Your struggle with your fellow citizens can be modelled as follows:



There are some nodes that we are sometimes able to prune. In each part, mark all of the terminal nodes such that **there exists a possible situation** for which the node **can be pruned**. In other words, you must consider **all** possible pruning situations. Assume that evaluation order is **left to right** and all V_i 's are **distinct**.

Note that as long as there exists ANY pruning situation (does not have to be the same situation for every node), you should mark the node as prunable. Also, alpha-beta pruning does not apply here, simply prune a sub-tree when you can reason that its value will not affect your final utility.

- (a)
- ☐ V_1
 - ☐ V_2
 - ☐ V_3
 - ☐ V_4
 - ☒ None

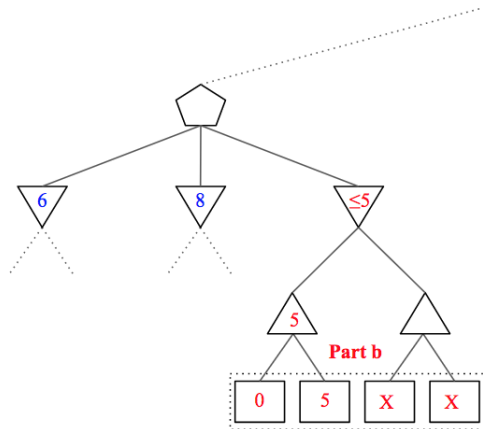
- (b)
- ☐ V_5
 - ☒ V_6
 - ☒ V_7
 - ☒ V_8
 - ☐ None

- (c)
- ☐ V_9
 - ☐ V_{10}
 - ☒ V_{11}
 - ☒ V_{12}
 - ☐ None

- (d)
- ☐ V_{13}
 - ☒ V_{14}
 - ☒ V_{15}
 - ☒ V_{16}
 - ☐ None

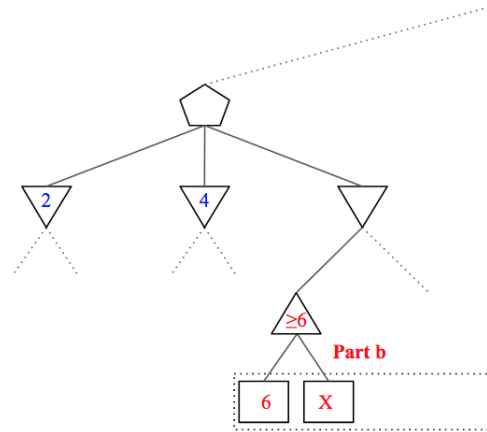
Part a:

For the left median node with three children, at least two of the childrens' values must be known since one of them will be guaranteed to be the value of the median node passed up to the final maximizer. For this reason, none of the nodes in part a can be pruned.



The value of this subtree will only get smaller.

The median node will **NOT** choose the value of this subtree. 6 is the median.



The value of this subtree will only get bigger.

If the value of this subtree is chosen by the minimizer*, it will **NOT** be chosen by the median node.

*It is possible that the median is the value of the subtree to the right that we haven't looked at yet

Part b (pruning V_7, V_8):

Let min_1, min_2, min_3 be the values of the three minimizer nodes in this subtree.

In this case, we may not need to know the final value min_3 . The reason for this is that we may be able to put a bound on its value after exploring only partially, and determine the value of the median node as either min_1 or min_2 if $min_3 \leq \min(min_1, min_2)$ or $min_3 \geq \max(min_1, min_2)$.

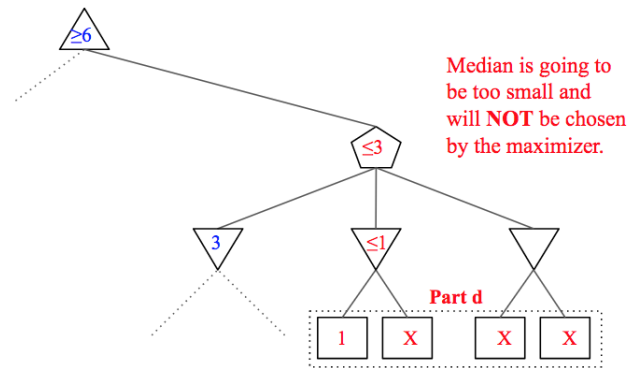
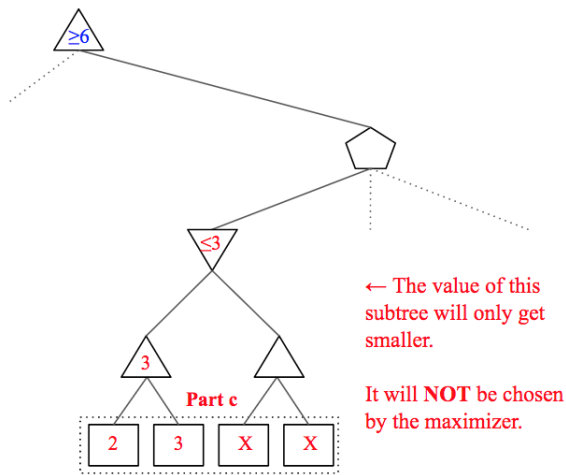
We can put an upper bound on min_3 by exploring the left subtree V_5, V_6 and if $\max(V_5, V_6)$ is lower than both min_1 and min_2 , the median node's value is set as the smaller of min_1, min_2 and we don't have to explore V_7, V_8 in Figure 1.

Part b (pruning V_6):

It's possible for us to put a lower bound on min_3 . If V_5 is larger than both min_1 and min_2 , we do not need to explore V_6 .

The reason for this is subtle, but if the minimizer chooses the left subtree, we know that $min_3 \geq V_5 \geq \max(min_1, min_2)$ and we don't need V_6 to get the correct value for the median node which will be the larger of min_1, min_2 .

If the minimizer chooses the value of the right subtree, the value at V_6 is unnecessary again since the minimizer never chose its subtree.



Part c (pruning V_{11}, V_{12}):

Assume the highest maximizer node has a current value $max_1 \geq Z$ set by the left subtree and the three minimizers on this right subtree have value min_1, min_2, min_3 .

In this part, if $min_1 \leq \max(V_9, V_{10}) \leq Z$, we do not have to explore V_{11}, V_{12} . Once again, the reasoning is subtle, but we can now realize if either $min_2 \leq Z$ or $min_3 \leq Z$ then the value of the right median node is for sure $\leq Z$ and is useless.

Only if both $min_2, min_3 \geq Z$ will the whole right subtree have an effect on the highest maximizer, but in this case the exact value of min_1 is not needed, just the information that it is $\leq Z$. Clearly in both cases, V_{11}, V_{12} are not needed since an exact value of min_1 is not needed.

We will also take the time to note that if $V_9 \geq Z$ we do have to continue the exploring as V_{10} could be even greater and the final value of the top maximizer, so V_{10} can't really be pruned.

Part d (pruning V_{14}, V_{15}, V_{16}):

Continuing from part c, if we find that $min_1 \leq Z$ and $min_2 \leq Z$ we can stop.

We can realize this as soon we explore V_{13} . Once we figure this out, we know that our median node's value must be one of these two values, and neither will replace Z so we can stop.

Q2. Games

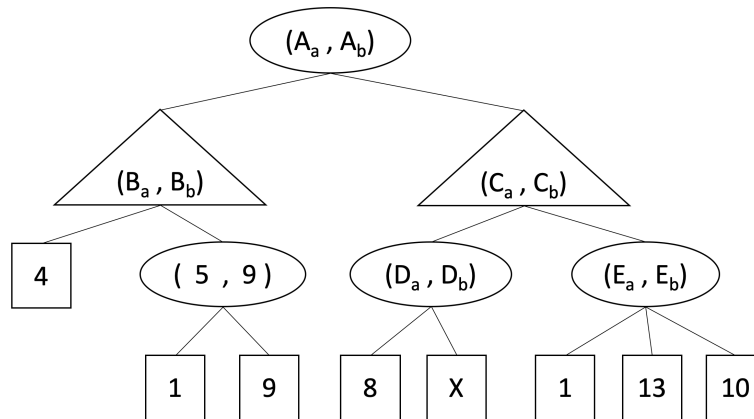
Alice is playing a two-player game with Bob, in which they move alternately. Alice is a maximizer. Although Bob is also a maximizer, Alice believes Bob is a minimizer with probability 0.5, and a maximizer with probability 0.5. Bob is aware of Alice's assumption.

In the game tree below, square nodes are the outcomes, triangular nodes are Alice's moves, and round nodes are Bob's moves. Each node for Alice/Bob contains a tuple, the left value being Alice's expectation of the outcome, and the right value being Bob's expectation of the outcome.

Tie-breaking: choose the left branch.

The left values are Alice's expectations, and are the only thing Alice can refer to when making decisions.

The right values are Bob's expectations, and they also accurately track the expected outcome of the game according to each choice of branching (regardless of it is Alice's or Bob's decision, since Bob has all the information). Hence the right values are accurate information about the game, and would be what Bob looks at when making his decisions. However, when it is Alice's turn to make decisions, Bob will think about how Alice would maximize the outcome w.r.t to what she believes, and he will update his expectations accordingly.



- (a) In the blanks below, fill in the tuple values for tuples (B_a, B_b) and (E_a, E_b) from the above game tree.

$(B_a, B_b) = ($ $,$ $)$

$(E_a, E_b) = ($ $,$ $)$

For a square node, its value v means the same to Alice and Bob, i.e., we can think of it as a tuple (v, v) .

The left value of Alice's nodes is the maximum of the left values of its children nodes, since Alice believes that the values of the nodes are given by left values, and it's her turn of action, so she will choose the largest value.

The right value of Alice's nodes is the right value from the child node that attains the maximum left value since Bob's expectation is consistent with how Alice will act.

So for a triangular node, its tuple is the same as its child that has the maximum left value.

The left value of Bob's nodes is the average of the maximum and minimum of the left values of its children nodes since Alice believes Bob is 50% possible to be adversarial and 50% possible to be friendly.

The right value of Bob's nodes is the maximum of the right values of the immediate children nodes since Bob would choose the branch that gives the maximum outcome during his turn.

So for a round node, $\text{left} = 0.5(\max(\text{children.left}) + \min(\text{children.left}))$, and $\text{right} = \max(\text{children.right})$.

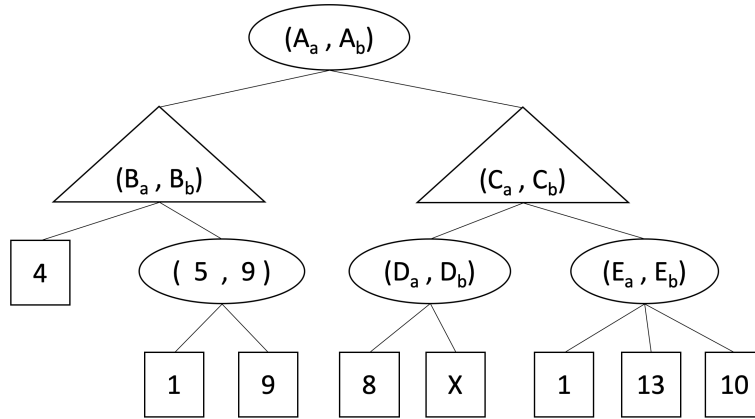
- (b) In this part, we will determine the values for tuple (D_a, D_b) .

(i) $D_a =$ ☐ 8 ☐ X ☐ $8+X$ ☒ $4+0.5X$ ☐ $\min(8, X)$ ☐ $\max(8, X)$

(ii) $D_b =$ ☐ 8 ☐ X ☐ $8+X$ ☐ $4+0.5X$ ☐ $\min(8, X)$ ☒ $\max(8, X)$

It's a round node, so $\text{left} = 0.5(\max(\text{children.left}) + \min(\text{children.left}))$, and $\text{right} = \max(\text{children.right})$.

Its children: $(8, 8)$ and (X, X) . So $\text{left} = 0.5(8+X) = 4+0.5X$, and $\text{right} = \max(8, X)$.



(The graph of the tree is copied for your convenience. You may do problem e on this graph.)

- (c) Fill in the values for tuple (C_a, C_b) below. For the bounds of X , you may write scalars, ∞ or $-\infty$.
If your answer contains a fraction, please write down the corresponding **simplified decimal value** in its place. (i.e., 4 instead of $\frac{8}{2}$, and 0.5 instead of $\frac{1}{2}$).

1. If $-\infty < X < \boxed{6}$, $(C_a, C_b) = (\boxed{7}, \boxed{13})$
2. Else, $(C_a, C_b) = (\boxed{4+0.5X}, \max(\boxed{8}, \boxed{X}))$

It's a triangular node, so its tuple is the same as its child that has the maximum left value.

Its children: $(4+0.5X, \max(8, X))$ and $(7, 13)$.

So if $4+0.5X < 7$, i.e. $-\infty < X < 6$, it's the same as child node $(7, 13)$, and otherwise it's $(4+0.5X, \max(8, X))$.

- (d) Fill in the values for tuple (A_a, A_b) below. For the bounds of X , you may write scalars, ∞ or $-\infty$.
If your answer contains a fraction, please write down the corresponding **simplified decimal value** in its place. (i.e., 4 instead of $\frac{8}{2}$, and 0.5 instead of $\frac{1}{2}$).

1. If $-\infty < X < \boxed{6}$, $(A_a, A_b) = (\boxed{6}, \boxed{13})$
2. Else, $(A_a, A_b) = (\boxed{4.5+0.25X}, \max(\boxed{9}, \boxed{X}))$

It's a round node, so $\text{left} = 0.5(\max(\text{children.left}) + \min(\text{children.left}))$, and $\text{right} = \max(\text{children.right})$.

Its children: $(5, 9)$ and node "Part (c)".

If $-\infty < X < 6$, these children are $(5, 9)$ and $(7, 13)$.

$\text{left} = 0.5(\max(\text{children.left}) + \min(\text{children.left})) = 0.5(5+7) = 6$

$\text{right} = \max(\text{children.right}) = \max(9, 13) = 13$.

Otherwise $(6 < X < +\infty)$, these children are $(5, 9)$ and $(4+0.5X, \max(8, X))$.

$\text{left} = 0.5(\max(\text{children.left}) + \min(\text{children.left})) = 0.5(5+4+0.5X) = 4.5 + 0.25X$

$\text{right} = \max(\text{children.right}) = \max(9, \max(8, X)) = \max(9, X)$.

- (e) When Alice computes the left values in the tree, some branches can be pruned and do not need to be explored. In the game tree graph on this page, put an 'X' on these branches. If no branches can be pruned, mark the "Not possible" choice below.

Assume that the children of a node are visited in left-to-right order and that you should not prune on equality.

● Not possible

It's impossible to determine the average of *min* and *max* until all children nodes are seen, so no pruning can be done for Alice. Leaving "Not possible" unmarked and no 'X' found in the graph is interpreted as 'no conclusion' and will not be given credit.

Introduction to Artificial Intelligence Midterm Review Search Solutions

Q1. Search

Each True/False question is worth 1 points. Leaving a question blank is worth 0 points. **Answering incorrectly is worth -1 points.**

- (a) Consider a graph search problem where for every action, the cost is at least ϵ , with $\epsilon > 0$. Assume the used heuristic is consistent.

- (i) [true or false] Depth-first graph search is guaranteed to return an optimal solution.
False. Depth first search has no guarantees of optimality. Further, it measures paths in length and not cost.
- (ii) [true or false] Breadth-first graph search is guaranteed to return an optimal solution.
False. Breadth first search has no guarantees of optimality unless the actions all have the same cost, which is not the case here.
- (iii) [true or false] Uniform-cost graph search is guaranteed to return an optimal solution.
True. UCS expands paths in order of least total cost so that the optimal solution is found.
- (iv) [true or false] Greedy graph search is guaranteed to return an optimal solution.
False. Greedy search makes no guarantees of optimality. It relies solely on the heuristic and not the true cost.
- (v) [true or false] A* graph search is guaranteed to return an optimal solution.
True, since the heuristic is consistent in this case.
- (vi) [true or false] A* graph search is guaranteed to expand no more nodes than depth-first graph search.

False. Depth-first graph search could, for example, go directly to a sub-optimal solution.

- (vii) [true or false] A* graph search is guaranteed to expand no more nodes than uniform-cost graph search.

True. The heuristic could help to guide the search and reduce the number of nodes expanded. In the extreme case where the heuristic function returns zero for every state, A* and UCS will expand the same number of nodes. In any case, A* with a consistent heuristic will never expand more nodes than UCS.

- (b) Let $h_1(s)$ be an admissible A* heuristic. Let $h_2(s) = 2h_1(s)$. Then:

- (i) [true or false] The solution found by A* tree search with h_2 is guaranteed to be an optimal solution.

False. h_2 is not guaranteed to be admissible since only one side of the admissibility inequality is doubled.

- (ii) [true or false] The solution found by A* tree search with h_2 is guaranteed to have a cost at most twice as much as the optimal path.

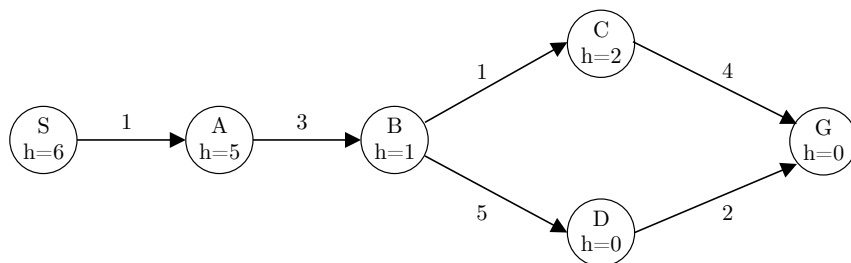
True. In A* tree search we always have that as long as the optimal path to the goal has not been found, a prefix of this optimal path has to be on the fringe. Hence, if a non-optimal solution is found, then at time of popping the non-optimal path from the fringe, a path that is a prefix of the optimal path to the goal is sitting on the fringe. The cost $\bar{g}$ of a non-optimal solution when popped is its f-cost. The prefix of the optimal path to the goal has an f-cost of $g + h_0 = g + 2h_1 \leq 2(g + h_1) \leq 2C^*$, with C^* the optimal cost to the goal. Hence we have that $\bar{g} \leq 2C^*$ and the found path is at most twice as long as the optimal path.

(iii) [*true* or *false*] The solution found by A* graph search with h_2 is guaranteed to be an optimal solution.

False. h_2 is not guaranteed to be admissible and graph search further requires consistency for optimality.

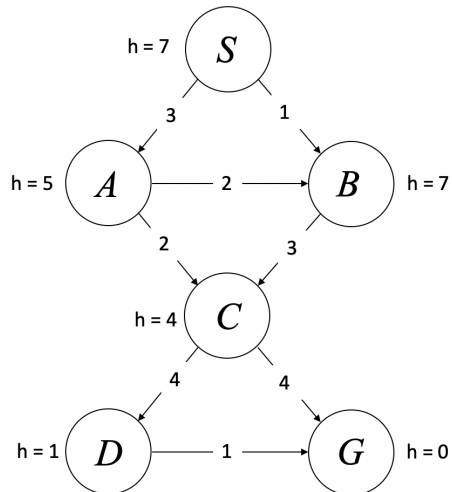
- (c) The heuristic values for the graph below are not correct. For which single state (S, A, B, C, D, or G) could you change the heuristic value to make everything admissible and consistent? What range of values are possible to make this correction?

State: Range:



Q2. Search Algorithms Potpourri

- (a) We will investigate various search algorithms for the following graph. Edges are labeled with their costs, and heuristic values h for states are labeled next to the states. S is the start state, and G is the goal state. In all search algorithms, assume ties are broken in alphabetical order.



- (i) Select all boxes that describe the given heuristic values.

☒ admissible ☒ consistent ☐ Neither

- (ii) Given the above heuristics, what is the order that the states are going to be expanded in, assuming we run A* graph search with the heuristic values provided.

Index	1	2	3	4	5	Not Expanded
S	☒	☐	☐	☐	☐	☐
A	☐	☒	☐	☐	☐	☐
B	☐	☐	☒	☐	☐	☐
C	☐	☐	☐	☒	☐	☐
D	☐	☐	☐	☐	☐	☒
G	☐	☐	☐	☐	☒	☐

- (iii) Assuming we run A* graph search with the heuristic values provided, what path is returned?

☐ $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$ ☐ $S \rightarrow A \rightarrow C \rightarrow G$ ☐ $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$
☒ $S \rightarrow B \rightarrow C \rightarrow G$ ☐ $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$ ☐ $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$
☐ $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$ ☐ None of the above

- (iv) Given the above heuristics, what is the order that the states are going to be expanded in, assuming we run greedy graph search with the heuristic values provided.

Index	1	2	3	4	5	Not Expanded
S	☒	☐	☐	☐	☐	☐
A	☐	☒	☐	☐	☐	☐
B	☐	☐	☐	☐	☐	☒
C	☐	☐	☒	☐	☐	☐
D	☐	☐	☐	☐	☐	☒
G	☐	☐	☐	☒	☐	☐

(v) What path is returned by greedy graph search?

- ☐ $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$
☒ $S \rightarrow A \rightarrow C \rightarrow G$
☐ $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$
☐ $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$
☐ None of the above

(b) Consider a complete graph, K_n , the undirected graph with n vertices where all n vertices are connected (there is an edge between every pair of vertices), resulting in $\binom{n}{2}$ edges. Please select the maximum possible depth of the resulting tree when the following **graph** search algorithms are run (assume any possible start and goal vertices).

	1	$\lceil \frac{n}{2} \rceil$	$n - 1$	$\binom{n}{2}$	None of the above
BFS	☒	☐	☐	☐	☐
DFS	☐	☐	☒	☐	☐

(c) Given two admissible heuristics h_A and h_B .

(i) Which of the following are guaranteed to also be admissible heuristics?

- ☐ $h_A + h_B$
☒ $\frac{1}{2}(h_A)$
☒ $\frac{1}{2}(h_B)$
☒ $\frac{1}{2}(h_A + h_B)$
☐ $h_A * h_B$
☒ $\max(h_A, h_B)$
☒ $\min(h_A, h_B)$

(ii) Consider performing A* **tree** search. Which is generally best to use if we want to expand the fewest number of nodes? **Note this was changed from graph to tree search during the exam**

- ☐ $h_A + h_B$
☐ $\frac{1}{2}(h_A)$
☐ $\frac{1}{2}(h_B)$
☐ $\frac{1}{2}(h_A + h_B)$
☐ $h_A * h_B$
☒ $\max(h_A, h_B)$
☐ $\min(h_A, h_B)$

(d) Consider performing tree search for some search graph. Let $depth(n)$ be the depth of search node n and $cost(n)$ be the total cost from the start state to node n . Let G_d be a goal node with minimum depth, and G_c be a goal node with minimum total cost.

(i) For iterative deepening (where we repeatedly run DFS and increase the maximum depth allowed by 1), mark all conditions that are guaranteed to be true for every node n that could be expanded during the search, or mark "None of the above" if none of the conditions are guaranteed.

- ☐ $cost(n) \leq cost(G_c)$
☐ $cost(n) \leq cost(G_d)$
☒ $depth(n) \leq depth(G_c)$
☒ $depth(n) \leq depth(G_d)$
☐ None of the above

When running iterative deepening we will explore all nodes of depth k , before we explore any nodes of depth $k + 1$. As a result will never explore any nodes that have depth greater than G_d because we would stop exploring once we reached G_d . This also means we would never explore any nodes with depth greater than G_c because G_c has to have depth greater than or equal to the minimum depth goal, G_d .

(ii) What is necessarily true regarding iterative deepening on any search tree?

- ☒ Complete as opposed to DFS tree search
☐ Strictly faster than DFS tree search
☐ Strictly faster than BFS tree search
☒ More memory efficient than BFS tree search

- ☐ A type of stochastic local search
- ☐ None of the above

Q3. They See Me Rolling (Search Problem)

Pacman buys a car to start Rolling in the Pac-City! But driving a car requires a new set of controls because he can now travel faster than 1 grid per turn (second). Instead of solely moving [North, South, East, West, Stop], Pacman's car has two distinct integers to control: throttle, and steering.

Throttle: $t_i \in \{1, 0, -1\}$, corresponding to {Gas, Coast, Brake}. This controls the **speed** of the car by determining its acceleration. The integer chosen here will be added to his velocity for the next state. For example, if Pacman is currently driving at 5 grid/s and chooses Gas he will be traveling at 6 grid/s in the next turn.

Steering: $s_i \in \{1, 0, -1\}$, corresponding to {Turn Left, Neutral, Turn Right}. This controls the **direction** of the car. For example, if he is facing North and chooses Turn Left he will be facing West in the next turn.

- (a) Suppose Pac-city has dimension m by n , but only $k < mn$ squares are legal roads. The speed limit of Pac-city is 3 grid/s. For this sub-part only, suppose Pacman is a law-abiding citizen, so $0 \leq v \leq 3$ at all time, and he only drives on legal roads.

- (i) Without any additional information, what is the **tightest upper bound** on the size of state space, if he wants to search a route (not necessarily the shortest) from his current location to anywhere in the city. Please note that your state space representation must be able to represent **all** states in the search space.

☐ $4mn$ ☐ $4k$ ☐ $12mn$ ☐ $12k$ ☐ $16mn$ ☒ $16k$ ☐ $48mn$ ☐ $48k$

Only legal grids count, so there are k legal position. At each legal position, there are 4 possible speed (0, 1, 2, 3), so a factor of 4 is multiplied. In addition, since change of direction depends on orientation of the car, another factor of 4 is multiplied. The size of state space is bounded by $k * 4 * 4 = 16k$

- (ii) What is the maximum branching factor? The answer should be an integer.

9

3 possible throttle inputs, and 3 possible steering inputs.

- (iii) Which algorithm(s) is/are guaranteed to return a path between two points, if one exists?

☐ Depth First Tree Search ☒ Breadth First Tree Search
☒ Depth First Graph Search ☒ Breadth First Graph Search

- (iv) Is Breadth First Graph Search guaranteed to return the path with the shortest grid distance?

☐ Yes ☒ No

The Breadth First Graph Search is guranteed to return the path with the shortest amount of time, because each edge here represent moving for 1 unit of time.

- (b) Now let's remove the constraint that Pacman follows the speed limit. Now Pacman's speed is limited by the mechanical constraints of the car, which is 6 grid/s, double the speed limit.

Pacman is now able to drive twice as fast on the route to his destination. How do the following properties of the search problem change as a result of being able to drive twice as fast?

- (i) Size of State Space:

☒ Increases ☐ Stays the same ☐ Decreases

At each legal position, there are 7 possible speed (0,1, 2, 3, 4, 5, 6), so a factor of 7 is multiplied. size of state space is now $k * 7 * 4 = 28k$

- (ii) Maximum Branching Factor:

☐ Increases ☒ Stays the same ☐ Decreases

Branching factor is independent of top speed

For the following part, **mark all choices that could happen on any graph**

(iii) The number of nodes expanded with **Depth** First Graph Search:

☐ Increases ☐ Stays the same ☐ Decreases

Anything can happen with Depth First Graph Search. For example, too fast can jump past an intersection that would lead to a shorter path to the goal

- (c) Now we need to consider that there are $p > 0$ police cars waiting at $p > 0$ distinct locations trying to catch Pacman riding dirty!! All police cars are stationary, but once Pacman takes an action which lands him in the same grid as one police car, Pacman will be arrested and the game ends.

Pacman wants to find a route to his destination, without being arrested. How do the following properties of the search problem change as a result of avoiding the police?

- (i) Size of State Space:

☐ Increases ☒ Stays the same ☐ Decreases

The size of statespace is still the same because all the factors in state space calculation is the same

- (ii) Maximum Branching Factor:

☐ Increases ☒ Stays the same ☐ Decreases

Branching factor is independent of police presense

For the following part, **mark all choices that could happen on any graph**

- (iii) Number of nodes expanded with **Breadth** First Graph Search:

☒ Increases ☒ Stays the same ☒ Decreases

Again anything could happen with BFS. Policemen could block out misleading paths to decrease the number of expansion. They could also not affect anything. They could also block the closest goal causing you to explore more nodes.

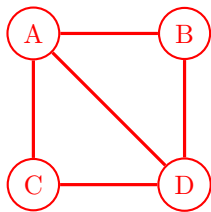
Introduction to Artificial Intelligence Midterm Review CSPs Solutions

Four people, A, B, C, and D, are all looking to rent space in an apartment building. There are three floors in the building, 1, 2, and 3 (where 1 is the lowest floor and 3 is the highest). Each person must be assigned to some floor, but it's ok if more than one person is living on a floor. We have the following constraints on assignments:

- A and B must not live together on the same floor.
- If A and C live on *the same* floor, they must both be living on floor 2.
- If A and C live on *different* floors, one of them must be living on floor 3.
- D must not live on the same floor as anyone else.
- D must live on a higher floor than C.

We will formulate this as a CSP, where each person has a variable and the variable values are floors.

- (a) Draw the edges for the constraint graph representing this problem. Use binary constraints only. You do not need to label the edges.



- (b) Suppose we have assigned $C = 2$. Apply forward checking to the CSP, filling in the boxes next to the values for each variable that are eliminated:

A	☒ 1	☐ 2	☐ 3
B	☐ 1	☐ 2	☐ 3
C		☐ 2	
D	☒ 1	☒ 2	☐ 3

- (c) Starting from the original CSP with full domains (i.e. without assigning any variables or doing the forward checking in the previous part), enforce arc consistency for the entire CSP graph, filling in the boxes next to the values that are eliminated for each variable:

A	☒ 1	☐ 2	☐ 3
B	☐ 1	☐ 2	☐ 3
C	☐ 1	☐ 2	☒ 3
D	☒ 1	☐ 2	☐ 3

- (d) Suppose that we were running local search with the min-conflicts algorithm for this CSP, and currently have the following variable assignments.

A	3
B	1
C	2
D	3

Which variable would be reassigned, and which value would it be reassigned to? Assume that any ties are broken alphabetically for variables and in numerical order for values.

The variable ☒ A will be assigned the new value ☐ 1
☐ B ☒ 2
☐ C ☐ 3
☐ D

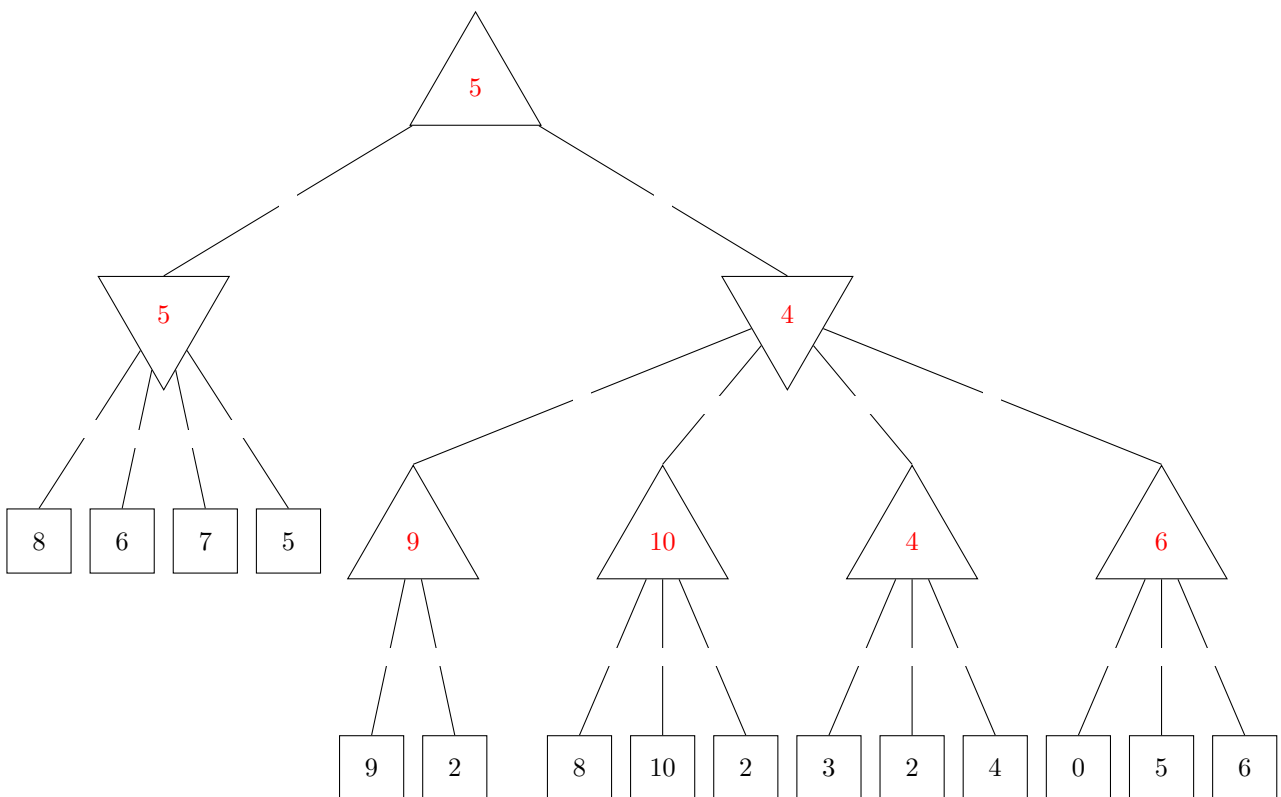
Introduction to Artificial Intelligence Midterm Review Games Solutions

Q1. Game Trees

The following problems are to test your knowledge of Game Trees.

(a) Minimax

The first part is based upon the following tree. Upward triangle nodes are maximizer nodes and downward nodes are minimizers. (small squares on edges will be used to mark pruned nodes in part (ii))



- (i) Complete the game tree shown above by filling in values on the maximizer and minimizer nodes.
- (ii) Indicate which nodes can be pruned by marking the edge above each node that can be pruned (you do not need to mark any edges below pruned nodes). In the case of ties, please prune any nodes that could not affect the root node's value. Fill in the bubble below if no nodes can be pruned.

☐ No nodes can be pruned

Edges that can be pruned: (parent-child) 10-2, 4-6, 6-0, 6-5, 6-6. So mark: 10-2, 4-6

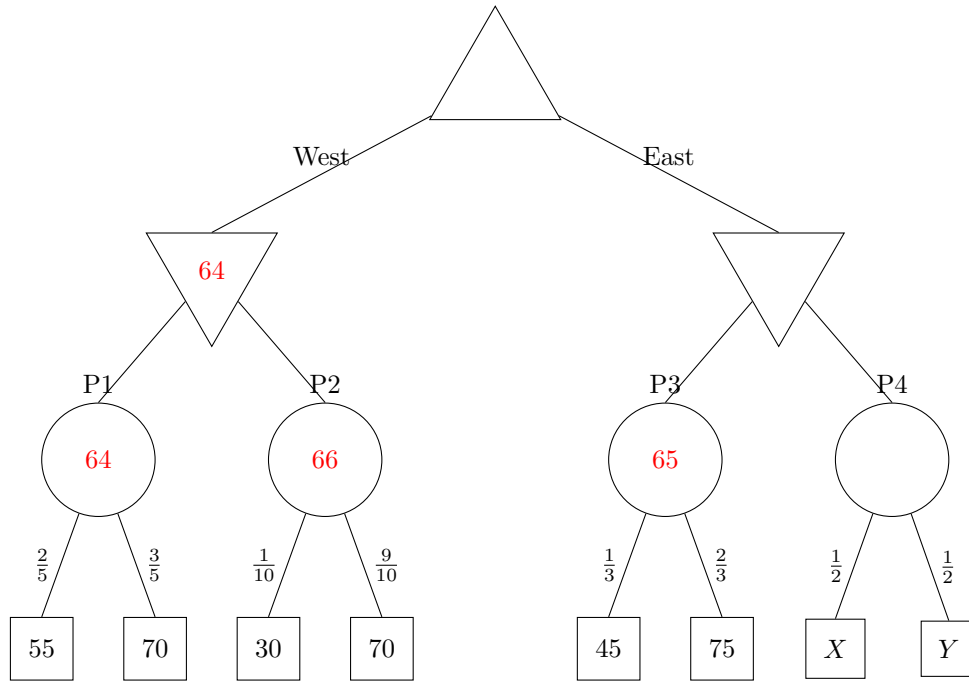
(b) Food Dimensions

The following questions are completely unrelated to the above parts.

Pacman is playing a tricky game. There are 4 portals to food dimensions. But, these portals are guarded by a ghost. Furthermore, neither Pacman nor the ghost know for sure how many pellets are behind each portal, though they know what options and probabilities there are for all but the last portal.

Pacman moves first, either moving West or East. After which, the ghost can block 1 of the portals available.

You have the following gametree. The maximizer node is Pacman. The minimizer nodes are ghosts and the portals are chance nodes with the probabilities indicated on the edges to the food. In the event of a tie, the left action is taken. Assume Pacman and the ghosts play optimally.



- (i) Fill in values for the nodes that do not depend on X and Y .
- (ii) What conditions must X and Y satisfy for Pacman to move East? What about to definitely reach the P4? Keep in mind that X and Y denote numbers of food pellets and must be **whole numbers**: $X, Y \in \{0, 1, 2, 3, \dots\}$.

To move East: $X + Y > 128$

To reach P4: $X + Y = 129$

The first thing to note is that, to pick A over B , $value(A) > value(B)$.

Also, the expected value of the parent node of X and Y is $\frac{X+Y}{2}$.

$$\Rightarrow \min(65, \frac{X+Y}{2}) > 64$$

$$\Rightarrow \frac{X+Y}{2} > 64$$

$$\text{So, } X + Y > 128 \Rightarrow value(A) > value(B)$$

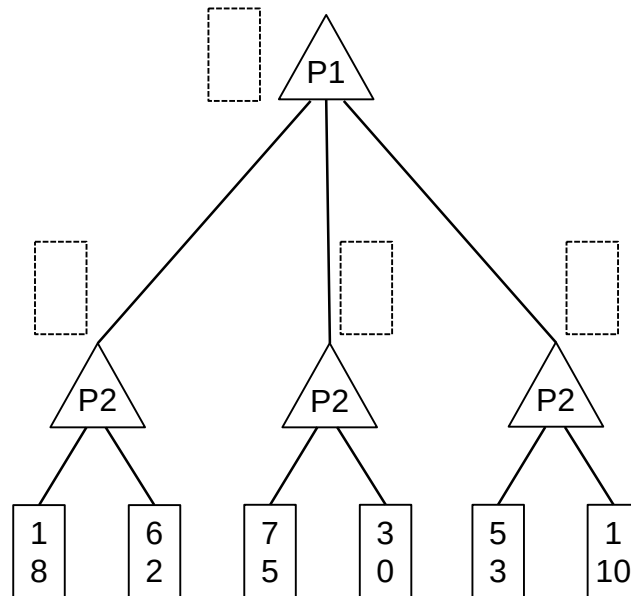
To ensure reaching X or Y , apart from the above, we also have $\frac{X+Y}{2} < 65$

$$\Rightarrow 128 < X + Y < 130$$

$$\text{So, } X, Y \in \mathbb{N} \Rightarrow X + Y = 129$$

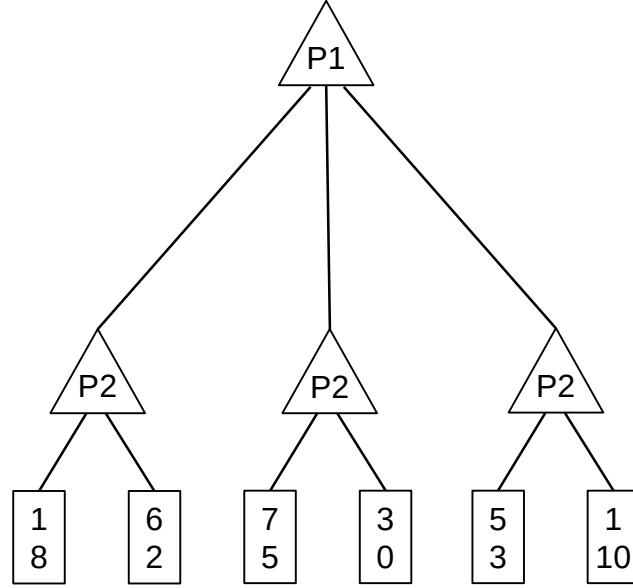
Q2. Games

For the following game tree, each player maximizes their respective utility. Let x, y respectively denote the top and bottom values in a node. Player 1 uses the utility function $U_1(x, y) = x$.



- (a) Both players know that Player 2 uses the utility function $U_2(x, y) = x - y$.
- Fill in the rectangles in the figure above with pair of values returned by each max node. **From top-down, left-right: (6, 2), (6, 2), (3, 0), (5, 3)**
 - You want to save computation time by using pruning in your game tree search. On the game tree above, put an 'X' on branches that do not need to be explored or simply write 'None'. Assume that branches are explored from left to right. **None.**

Figure repeated for convenience



- (b) Now assume Player 2 changes their utility function based on their mood. The probabilities of Player 2's utilities and mood are described in the following table. Let M, U respectively denote the mood and utility function of Player 2.

			$M = happy$	$M = mad$
$P(M = happy)$	$P(M = mad)$	$P(U_2(x, y) = -x \mid M)$	c	f
a	b	$P(U_2(x, y) = x - y \mid M)$	d	g
		$P(U_2(x, y) = x^2 + y^2 \mid M)$	e	h

- (i) Calculate the maximum expected utility of the game for Player 1 in terms of the values in the game tree and the tables. It may be useful to record and label your intermediate calculations. You may write your answer in terms of a max function.

We first calculate the new probabilities of each utility function as follows.

$P(U_2(x, y) = -x)$	$P(U_2(x, y) = x - y)$	$P(U_2(x, y) = x^2 + y^2)$
$ac + bf$	$ad + bg$	$ae + bh$

$$EU(\text{Left Branch}) = (ac + bf)(1) + (ad + bg)(6) + (ae + bh)(1)$$

$$EU(\text{Middle Branch}) = (ac + bf)(3) + (ad + bg)(3) + (ae + bh)(7)$$

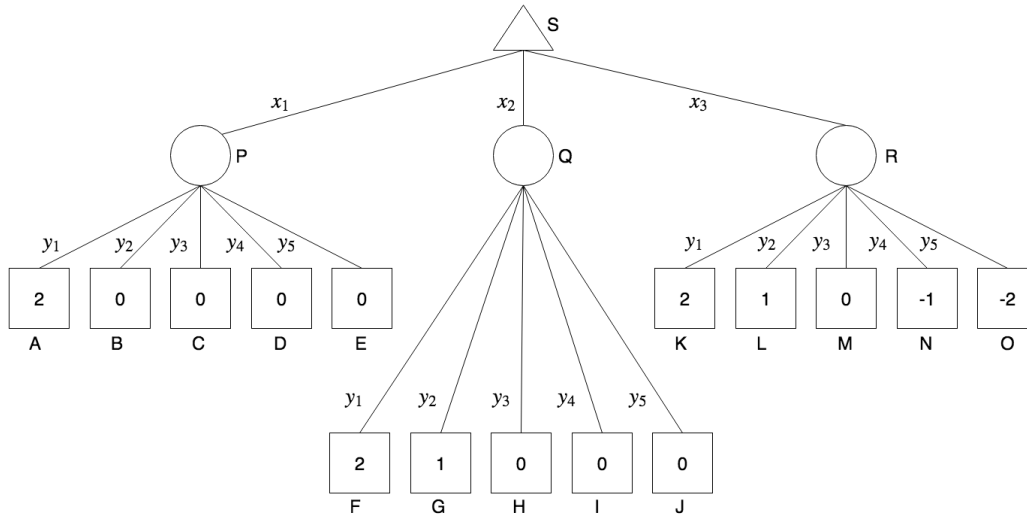
$$EU(\text{Right Branch}) = (ac + bf)(1) + (ad + bg)(5) + (ae + bh)(1)$$

$$MEU(\phi) = \max((ac + bf)(1) + (ad + bg)(6) + (ae + bh)(1), (ac + bf)(3) + (ad + bg)(3) + (ae + bh)(7), (ac + bf)(1) + (ad + bg)(5) + (ae + bh)(1))$$

Q3. Bike Bidding Battle

Alyssa P. Hacker and Ben Bitdiddle are bidding in an auction at Stanley University for a bike. Alyssa will either bid x_1 , x_2 , or x_3 for the bike. She knows that Ben will bid y_1 , y_2 , y_3 , y_4 , or y_5 , but she does not know which. All bids are nonnegative.

- (a) Alyssa wants to maximize her payoff given by the expectimax tree below. The leaf nodes show Alyssa's payoff. The nodes are labeled by letters, and the edges are labeled by the bid values x_i and y_i . The maximization node S represents Alyssa, and the branches below it represent each of her bids: x_1 , x_2 , x_3 . The chance nodes P, Q, R represent Ben, and the branches below them represent each of his bids: y_1 , y_2 , y_3 , y_4 , y_5 .



- (i) Suppose that Alyssa believes that Ben would bid any bid with equal probability. What are the values of the chance (circle) and maximization (triangle) nodes?

1. Node P 0.4
2. Node Q 0.6
3. Node R 0
4. Node S 0.6

- (ii) Based on the information from the above tree, how much should Alyssa bid for the bike?

☐ x_1 ☒ x_2 ☐ x_3

- (b) Alyssa does expectimax search by visiting child nodes from left to right. Ordinarily expectimax trees cannot be pruned without some additional information about the tree. Suppose, however, that Alyssa knows that the leaf nodes are ordered such that payoffs are non-increasing from left to right (the leaf nodes of the above diagram is an example of this ordering). Recall that if node X is a child of a maximizer node, a child of node X may be pruned if we know that the value of node X will never be $>$ some threshold (in other words, it is $\leq$ that threshold). Given this information, if it is possible to prune any branches from the tree, mark them below. Otherwise, mark "None of the above."

☐ A ☐ B ☐ C ☐ D ☐ E ☐ F ☐ G ☐ H
☐ I ☐ J ☐ K ☐ L ☐ M ☒ N ☒ O ☐ None of the above

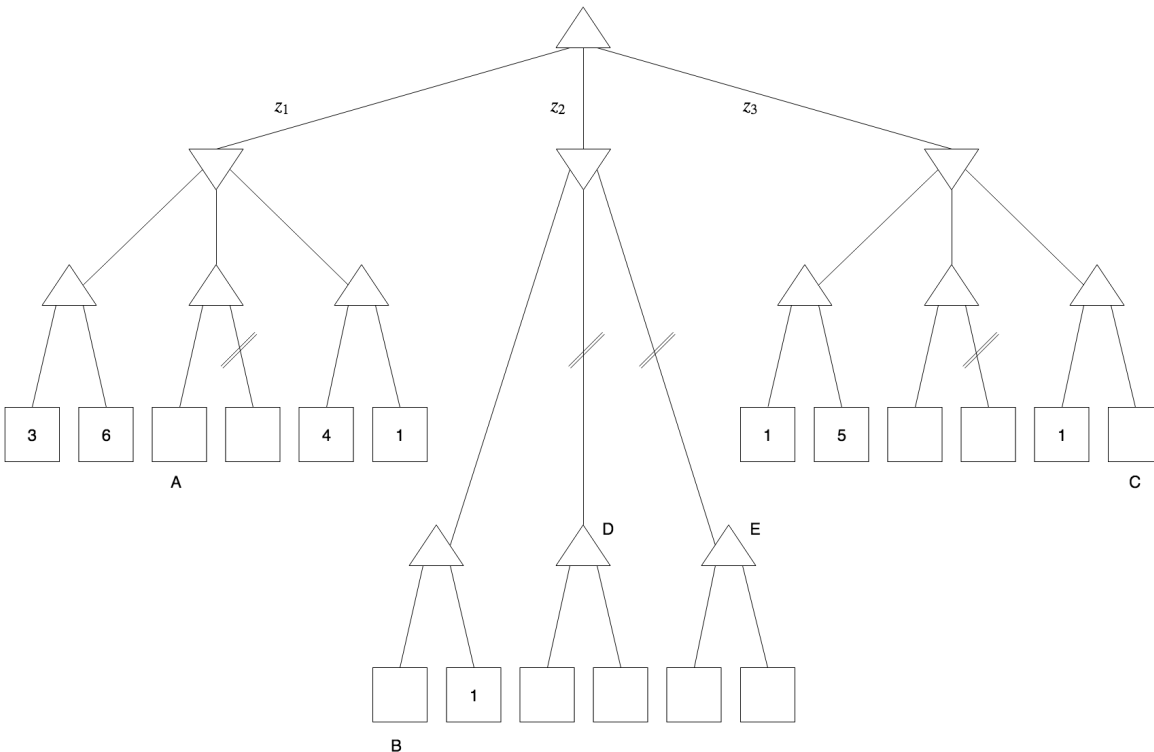
To prune the children of a chance node in an expectimax tree, Alyssa would need to keep track of a threshold on the value of the chance node: if at some point while searching left to right, she realizes that the value of the chance node will never be higher than its threshold, she can prune the remaining children of the chance node.

Alyssa needs to search the entire left subtree because she does not have a threshold against which to compare the value of P.

When Alyssa searches the center subtree, she knows that the maximizing node will only consider taking action x_2 if the value of Q is higher than the value of P, which is 0.4. If at some point Alyssa realizes that the value of Q will never be higher than 0.4, she can prune the remaining children. After exploring node G, Alyssa knows that nodes H, I, and J are ≤ 1 , which means that the value of node Q is at most 1.2. After exploring node H, Alyssa knows that nodes I and J are ≤ 0 , which means that the value of node Q is at most 0.6. After exploring node I, Alyssa knows that node J is ≤ 0 , which means that the value of node Q is at most 0.6. This is not enough information to prune any of the nodes in the center subtree because at no point does Alyssa know for sure that the value of Q is ≤ 0.4 .

When Alyssa searches the right subtree, if at some point Alyssa realizes that the value of R will never be higher than 0.6, then she can prune the remaining children of R. After exploring node L, Alyssa knows that the nodes M, N, and O are ≤ 1 , which means that the value of node R is at most 1.2. After exploring node M, Alyssa knows that nodes N and O are ≤ 0 , which means that the value of node Q is at most 0.6. At this point, Alyssa can prune nodes N and O because they can only make the value of R lower than the value of Q.

- (c) Unrelated to parts (a) and (b), consider the minimax tree below. whose leaves represent payoffs for the maximizer. The crossed out edges show the edges that are pruned when doing naive alpha-beta pruning visiting children nodes from left to right. Assume that we prune on equalities (as in, we prune the rest of the children if the current child is $\leq \alpha$ (if the parent is a minimizer) or $\geq \beta$ (if the parent is a maximizer)).



Fill in the inequality expressions for the values of the labeled nodes A and B. Write ∞ and $-\infty$ if there is no upper or lower bound, respectively.

$$1. \quad \boxed{6} \leq A \leq \boxed{\infty}$$

$$2. \quad \boxed{-\infty} \leq B \leq \boxed{4}$$

- (d) Suppose node B took on the largest value it could possibly take on and still be consistent with the pruning scheme above. After running the pruning algorithm, we find that the values of the left and center subtrees have the same minimax value, both 1 greater than the minimax value of the right subtree. Based on this information, what is the numerical value of node C?

☐ 1
 ☐ 2
 ☒ 3
 ☐ 4
 ☐ 5
 ☐ 6
 ☐ 7
 ☐ 8
 ☐ 9
 ☐ 10

- (e) For which values of nodes D and E would choosing to take action z_2 be *guaranteed* to yield the same payoff as action z_1 ? Write ∞ and $-\infty$ if there is no upper or lower bound, respectively (this would correspond to the case where nodes D and E can be any value).

$$1. \quad \boxed{4} \leq D \leq \boxed{\infty}$$

$$2. \quad \boxed{4} \leq E \leq \boxed{\infty}$$

When doing naive alpha-beta pruning, the values propagated up to the parent nodes are not necessarily exact, but rather bounds. If $D < 4$ or $E < 4$, then the true minimax value of the center subtree is less than the true minimax value of the left subtree.